



PROGRAMAÇÃO

Engenharia Informática, 1º ano, Ano Letivo 2016/2017

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

OBJECTIVOS

- ❑ Conhecer e utilizar os conceitos fundamentais do paradigma da programação orientada aos objectos;
- ❑ Desenvolver e implementar aplicações em linguagem JAVA que suportam este paradigma;
- ❑ Utilizar do ambiente de programação Netbeans.

PROGRAMA - RESUMO

- ❑ Introdução à linguagem de programação JAVA
- ❑ Programação Orientada aos Objectos em JAVA
- ❑ Os componentes GUI da linguagem JAVA
- ❑ JDBC

REFERÊNCIAS

- ❑ Thinking in Java, Bruce Eckel
- ❑ Oracle Technology Network – Java,
<http://www.oracle.com/technetwork/java/index.html>
- ❑ Java – How to Program, 10th edition, H.M.Deitel and P.J.Deitel, Pearson Education International – Prentice Hall
- ❑ NetBeans Community (<http://www.netbeans.org>)

□ Avaliação contínua

1. Construção de portefólio com exercícios práticos (20%) *
2. Frequência prática, em data a designar (40%)
3. Frequência, época de avaliações, marcado pela direção da Escola: teste escrito (40%)

□ Avaliação por exame final na Época Normal, Época de Recurso ou Época Especial:

1. Exame: teste escrito 60% + Parte Prática: 40% (Pode optar pela nota da frequência prática)

□ Avaliação contínua (Ponto 1)

- * Construção de portefólio individual com exercícios práticos (20%)
 - Exercício resolvido na aula do dia proposto 0 - 100%
 - Exercício completo (enunciado diferente ao proposto em sala de aula), resolvido no prazo máximo de **3 dias** a contar da data proposta: 0 - 100%
 - Exercício completo, resolvido no prazo máximo de **3 dias** a contar da data proposta: 0 - 75%
 - Qualquer outra situação: 0



IPG

Politécnico
da Guarda

Escola Superior
de Tecnologia e Gestão

LINGUAGEM JAVA

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

HISTÓRIA DO JAVA

- ❑ Com a revolução dos microprocessadores, no início dos anos 90, a Sun Microsystems criou um projecto de investigação de nome *Green*, com a intuito de a utilizar em diferentes dispositivos electrónicos.
- ❑ Deste projecto resultou uma linguagem denominada *Oak*, desenvolvida a partir da linguagem C++.
- ❑ O nome Java surgiu depois da equipa estar reunida num café.

HISTÓRIA DO JAVA

- ❑ O projecto Green atravessou grandes dificuldades e quase foi cancelado.
- ❑ No entanto, em 1993 a World Wide Web ganha um enorme popularidade.
- ❑ A equipa da Sun reconhece as enormes potencialidades da linguagem Java para a Web.

HISTÓRIA DO JAVA

- Em Maio de 1995, a Sun apresenta formalmente a linguagem Java.

LINGUAGEM JAVA

□ A linguagem de programação de alto-nível Java é caracterizada por:

- Simples
- Orientada aos objectos
- Distribuída
- Multithread
- Dinâmica
- Independente da arquitectura
- Portável
- Elevada performance
- Robusta
- Segura

TECNOLOGIA JAVA

- ❑ A ideia forte associada à linguagem Java é:

“Escrever código uma vez e executável em qualquer parte.”

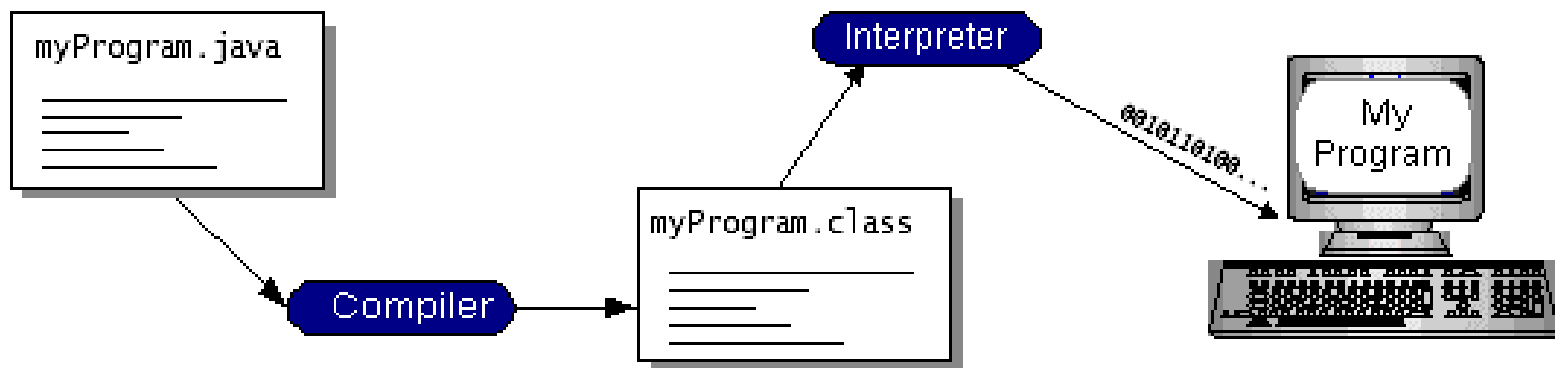
- ❑ Para que seja possível, o código fonte das aplicações escritas em Java tem de ser isolado dos sistemas operativos das máquinas e dos dispositivos de hardware.

TECNOLOGIA JAVA

- ❑ Para que tal seja possível, o código não pode ser compilado directamente para código nativo das máquinas, mas antes para uma representação especial, neutra, designada de **bytecode**.
- ❑ Em seguida, este código será interpretado sobre o ambiente particular de cada máquina.
- ❑ Em cada máquina, que se pretende executar programas Java, existirá um interpretador particular de bytecode.

TECNOLOGIA JAVA

- ❑ Este *runtime-engine*, ou motor de execução, designa-se por **Java Virtual Machine** (JVM).
- ❑ O JVM recebe **bytecode** e transforma-o em instruções executáveis na máquina particular.

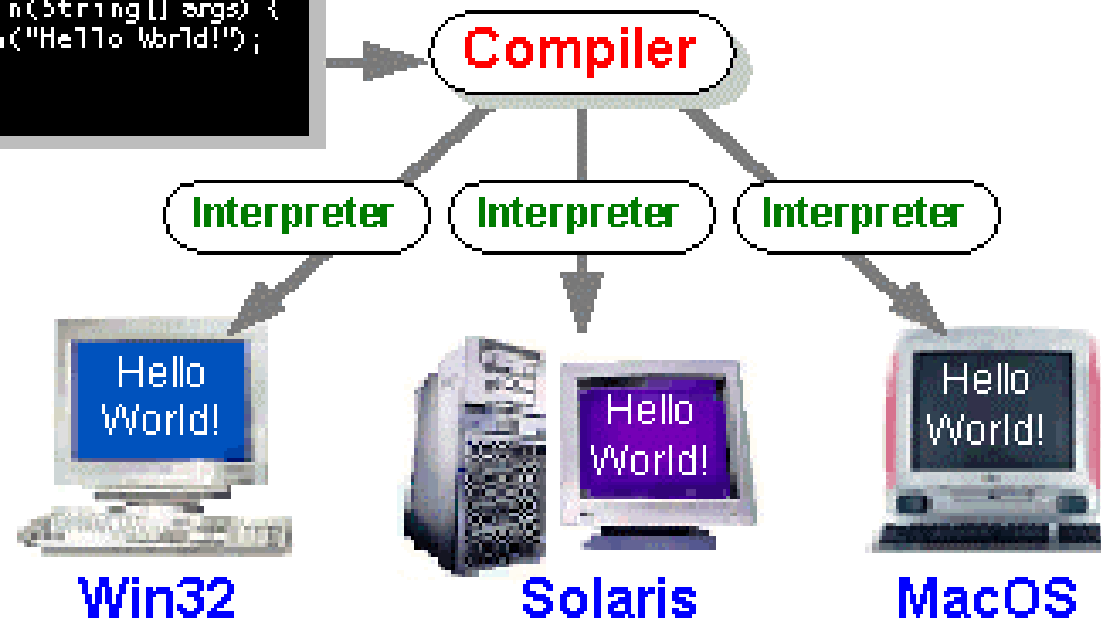


TECNOLOGIA JAVA

Java Program

```
class HelloWorldApp {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

HelloWorldApp.java



O KIT DE DESENVOLVIMENTO

- ❑ Em <http://www.oracle.com/technetwork/java/javase/downloads/index.html>, encontramos tudo o que necessitamos sobre Java.
- ❑ O kit de desenvolvimento contém:
 - ❑ O compilador: javac.exe
 - ❑ O interpretador: java.exe
 - ❑ O Appletviewer
 - ❑ Documentação
- ❑ Instalar uma das versões de desenvolvimento JDK (última versão: Java SE 8 Update 121 – 17/01/2017).

O KIT DE DESENVOLVIMENTO

❑ Editor

- ❑ Editor de texto simples, como o Wordpad; ou
- ❑ Integrated Development Environments (IDE)
 - ❑ Como por exemplo o **Netbeans** (www.netbeans.org)

NetBeans 8.2, em 1 de Março de 2017

 - ❑ jEdit (www.jedit.org)
 - ❑ Eclipse (www.eclipse.org)
 - ❑ Jbuilder (www.borland.com)
 - ❑ jCreator (www.jcreator.com)
 - ❑ BlueJ (www.bluej.org)
 - ❑ jGrasp (www.jgrasp.org)

PROGRAMAS EM JAVA

- ❑ Os programas em Java são constituídos por várias peças designadas por **classes**.
- ❑ As classes são constituídas por, outras peças, designadas por **métodos**.
- ❑ Os métodos executam tarefas e devolvem informação.

PROGRAMAS EM JAVA

- ❑ Os programadores podem criar cada uma destas peças de que necessitam para formar um programa em Java.
- ❑ Em Java temos o trabalho facilitado, pois **Java Class Libraries** contem um vasto conjunto de classes que podemos utilizar.

PROGRAMAS EM JAVA

- A este conjunto de classes: *Java Class Libraries*, é também designado por:

Java API

(Application Programming Interfaces)

PROGRAMAS EM JAVA

- ❑ Podemos dividir a aprendizagem do Java em duas etapas:
 1. A linguagem Java, que permitirá a criar a nossas próprias classes;
 2. Java Classes Libraries.

PROGRAMAS EM JAVA

□ Primeiro programa em Java

```
public class OlaMundo
{
    public static void main(String args[])
    {
        System.out.println("Olá Mundo!");
    }
}
```

- ❑ Utilizar o Netbeans
 - ❑ Operações básicas
 - ❑ Criar um projecto
 - ❑ Criar uma aplicação Java
 - ❑ Escrever o exemplo “OlaMundo.java”
 - ❑ Compilar e Executar

LINGUAGEM JAVA

❑ Boas práticas

- ❑ Iniciar os nomes das classes por letra maiúscula, por exemplo: OlaMundo, Aluno
- ❑ Avanços (*indent*) na escrita do código.

❑ Erros comuns:

- ❑ O Java é *case sensitive*. A não utilização dos identificadores exactamente como foram definidos, letras maiúsculas e minúsculas.

VARIÁVEIS E TIPOS DE DADOS

- ❑ Os nomes são fundamentais para a programação, eles são utilizados para especificar um grande número de coisas diferentes.
- ❑ De acordo com as regras de sintaxe do Java: os nomes são um sequência de um ou mais caracteres, que começam por uma letra e podem conter letras, dígitos ou underscore “_”.

VARIÁVEIS E TIPOS DE DADOS

- ❑ Alguns exemplos de nome válidos:
 - ❑ N
 - ❑ n
 - ❑ preco_unitário (snake_case ou underscore_case)
 - ❑ precoUnitario (camelCase, mais utilizado em Java)
 - ❑ X21
 - ❑ HelloWorld
 - ❑ codigo_Postal, codigoPostal
 - ❑ cod_iva_20, codIva20

VARIÁVEIS E TIPOS DE DADOS

- ❑ Caracteres maiúsculos e minúsculos são considerados diferentes, pelo que os nomes seguintes são considerados todos diferentes:
 - ❑ HelloWorld
 - ❑ helloworld
 - ❑ HELLOWORLD
 - ❑ hElloWorLD

VARIÁVEIS E TIPOS DE DADOS

- ❑ Existem um determinado número de palavras que não podem ser utilizadas por estarem reservadas à utilização pelo Java, são designadas de palavras reservadas.
- ❑ Exemplos de palavras reservadas:
 - ❑ public, class, main, static,
 - ❑ if, else, while, etc...

VARIÁVEIS E TIPOS DE DADOS

- ❑ Existem, também, nomes compostos por exemplo: **System.out.println**
- ❑ Estes são constituídos por um conjunto de nomes separados por “.” (ponto).
- ❑ A ideia é que os nomes compostos formam uma espécie de caminho, ou que as coisas contêm outras coisa.
- ❑ Por exemplo: System.out.println, qualquer coisa de nome **System**, contém qualquer coisa de nome **out**, e qualquer coisa de nome **out** contém **println**.

VARIÁVEIS E TIPOS DE DADOS

- ❑ Iremos utilizar o termo identificador para referir qualquer tipo de nome simples ou composto para citar qualquer coisa em Java.
- ❑ Os programas manipulam dados que estão armazenados em memória. Em linguagem máquina, os dados podem ser acedidos mediante o endereço numérico da sua localização em memória.
- ❑ Nas linguagens de alto nível os nomes são utilizados para substituir os números para fazer referência aos dados.

VARIÁVEIS E TIPOS DE DADOS

- ❑ É trabalho do computador saber a localização dos dados em memória através da identificação de um nome.
- ❑ Os nomes que são utilizados para representar dados em memória designamos de variáveis.
- ❑ Em Java, para atribuir dados a uma variável utilizamos o sinal de atribuição:



- ❑ Sintaxe:

variável = expressão

- ❑ Exemplos de atribuição:
 - valor = 8;
 - total = 2 * valor + taxa;

VARIÁVEIS E TIPOS DE DADOS

□ Tipos primitivos

Tipo	Valores	Default	Bits	Gama
boolean	true, false	false	1	-----
char	caracteres unicode	\u0000	16	\u0000 a \uFFFF
byte	inteiro c/ sinal	0	8	-128 a 127
short	inteiro c/ sinal	0	16	-32768 a 32767
int	inteiro c/ sinal	0	32	-2147483648 a 2147483647
long	inteiro c/ sinal	0	64	-1E+20 a 1E+20
float	IEEE 754 FP	0.0	32	+/-3.4E+38 a +/-1.4E-45
double	IEEE 754 FP	0.0	64	+/-1.8E+308 a +/-5E-324

VARIÁVEIS E TIPOS DE DADOS

- ❑ Uma variável só pode ser usada depois de declarada.
- ❑ A instrução de declaração de variável é:

Tipo_de_dados nome_da_variável;

- ❑ Exemplos:
 - ❑ `int numeroDeEstudantes;`
 - ❑ `String nome;`
 - ❑ `double x, y;`
 - ❑ `boolean isFim;`
 - ❑ `char primeiraLetra, ultimaLetra;`

VARIÁVEIS E TIPOS DE DADOS

- ❑ Mais exemplos de declaração de variáveis:

- ❑ `int x;`
- ❑ `int x = 10;`
- ❑ `int x=20, y, z=30;`
- ❑ `char um = '1';`
- ❑ `char c='A', newline = '\n';`
- ❑ `boolean fim, fechado = true;`
- ❑ `byte b1= 10;`
- ❑ `long raio = -1.7E+5;`
- ❑ `double d, j =0.000000123;`
- ❑ `double pi = 3.14159273269;`

CONSTANTES

- ❑ Declaração de constantes:
 - ❑ Semelhante à declaração de variáveis mas com a inclusão do atributo *final*.

final float pi = 3.141559273269;

CAST

- ❑ Conversão entre tipo: casting
 - ❑ Este operador é usado para converter valores de um dado tipo à sua direita para os valores do dado tipo à sua esquerda, desde que a compatibilidade seja possível.
 - ❑ `int x =2;`
 - ❑ `float f;`
 - ❑ `f = (float) 3 / x;`

TIPOS REFERENCIADOS

❑ Objectos e arrays

- ❑ Em Java, tipos não-primitivos são associados a objectos e a arrays.
- ❑ Estes tipos são, também, designados de tipos de referência.
- ❑ Os seus representantes são tratados por referência, ou seja, através de uma variável que contendo o seu endereço, dá acesso indirecto ao seu valor.

TIPOS REFERENCIADOS

❑ Por exemplo:

```
Ponto ponto1 = new Ponto(10,20);
```

❑ A variável *ponto1* é associada a um objecto da classe *Ponto*.

TIPOS REFERENCIADOS

□ Arrays

- Os arrays em Java são entidades referenciadas mas não são objectos.
- O arrays são criados dinamicamente, ou seja, em tempo de execução, e o seu espaço automaticamente reaproveitado quando deixam de estar referenciados.

TIPOS REFERENCIADOS

❑ Exemplos de declaração:

- ❑ `int lista[] = {1,2,3,4,5,6,7,8,9};`
- ❑ `String [] cores = {"vermelho","verde","azul"};`
- ❑ `int lista[] = new int[20];` // array com 20 elementos de nome *lista*
- ❑ `byte[] pixels = new byte[600*800];`
- ❑ `String []texto = new String[200];`
- ❑ `int [][] tabela = new int[30][20];` // array bidimensional
- ❑ `int [][] matriz = { {1,2},{10,20},{100,200,300} };`

TIPOS REFERENCIADOS

- ❑ Acesso aos elementos de um array
 - ❑ O acesso ao *array* faz-se através de um índice inteiro, desde **0** (1^a. posição do *array*) até **length-1** (última posição).
 - ❑ `int [] a = new int[20];`
 - ❑ `byte[] pixels = new byte[600*800];`
 - ❑ `int x, y;`
 - ❑ `x = a[0]; y = a[a.length-1];`
 - ❑ `b = pixels[12*24];`

OPERADORES

❑ Operadores sobre tipos primitivos

Operador	Tipo Operando(s)	Associatividade	Operação
++	aritméticos	D	pré/pós incremento
--	aritméticos	D	pré/pós incremento
+ -	aritméticos	D	sinal; unário
~	integral	D	complemento de bits
!	booleano	D	negação
(tipo)	qualquer	E	conversão (casting)
* / %	aritméticos	E	mult., div., resto
+ -	aritméticos	E	soma e subtracção
+	strings	E	concatenação
<<	integral	E	desl. de bits para esq.
>>	integral	E	desl. de bits para dir.
>>>	integral	E	desl. de bits com 0
<<=	aritméticos	E	comparação
>>=	aritméticos	E	comparação
instanceof	objecto, tipo	E	teste do tipo
==	primitivos	E	igual valor

OPERADORES

❑ Operadores sobre tipos primitivos

Operador	Tipo Operando(s)	Associatividade	Operação
!=	primitivos	E	valor diferente
&	integral	E	E de bits
&	booleano	E	E lógico
^	integral	E	OUEXEC de bits
^	booleano	E	OUEXEC lógico
	integral	E	OU de bits
	booleano	E	OU lógico
&&	booleano	E	E condicional
	booleano	E	OU condicional
?:	bool., qq, qq	E	
=	variável, qq	D	atribuição
*= /= %=	variável, qq	D	atribuição com oper.
+= -=	variável, qq	D	atribuição com oper.
<<= >>=	variável, qq	D	atribuição com oper.
>>>= &=	variável, qq	D	atribuição com oper.
^= =	variável, qq	D	atribuição com oper.

ASSOCIATIVIDADE

□ Exemplo:

Operador: - (subtração) – Associatividade esquerda

$w = x - y - z$	$x = 1, y = 2, z = 3$
$wD = x - (y - z)$	$wD = 1 - (2 - 3) = 1 + 1 = 2$
$wE = (x - y) - z$	$wE = (1 - 2) - 3 = -1 - 3 = -4$

Operador: = (atribuição) – Associatividade direita

$w = x = y = z$	$x = 1, y = 2, z = 3$
$wD = x = (y = z)$	$wD = x = (y = 3) = x = 3 = 3$
$wE = (x = y) = z$	Erro!

ESTRUTURAS DE CONTROLO

❑ if (condição booleana)

Instrução se verdade;

Exemplo:

```
if (nota >= 60)
```

```
    System.out.println("Aprovado");
```

ESTRUTURAS DE CONTROLO

❑ if (condição booleana)

Instrução se verdade;

❑ else

Instrução se falso;

Exemplo:

```
if (nota >= 60)
```

```
    System.out.println("Aprovado");
```

```
else
```

```
    System.out.println("Reprovado");
```

ESTRUTURAS DE CONTROLO

```
if ( condição booleana )  
    Instrução 1;  
else if (condição booleana )  
    Instrução 2;  
else if ( condição booleana )  
    Instrução 3;  
else  
    Instrução 4;
```

Exemplo:

```
if (nota >= 90)  
    System.out.println("A");  
else if (nota >= 80)  
    System.out.println("B");  
else if (nota >= 70)  
    System.out.println("C");  
else if (nota >= 60)  
    System.out.println("D");  
else  
    System.out.println("F");
```

ESTRUTURAS DE CONTROLO

❑ Operador condicional

Condição booleana ? Instrução se verdade : Instrução se falso;

Exemplo:

```
System.out.println( nota >= 60 ? “Aprovado” : “Reprovado” );
```


ESTRUTURAS DE CONTROLO

□ Estrutura de repetição: while

□ while (condição)

Instrução

□ Exemplo:

`i = 1;`

`while ( i <= 20 )`

`i = i * 2;`

ESTRUTURAS DE CONTROLO

- ❑ Estrutura de repetição: do / while

- ❑ do {

 - Instrução;

- ❑ } while (condição);

- ❑ Exemplo:

 - $i = 1;$

 - do {

 - $i = i * 2;$

 - } while ($i \leq 20$);

ESTRUTURAS DE CONTROLO

❑ Estrutura de repetição: for

for (valor inicial; condição de paragem; incremento)

❑ Exemplo:

```
for ( i = 1; i <=20; i++)  
    System.out.println(i);
```

I/O BÁSICO

Exemplo:

```
String s = new String();  
int n;  
char c;  
  
System.out.print("Valor : ");  
do{  
    c = (char)System.in.read();  
    s += c;  
} while (c!='\n');  
s = s.substring(0,s.length()-1);  
n = Integer.valueOf(s).intValue();
```

Em Java, qualquer método que possa produzir um erro sério deve ser protegido por algum método de detecção de erros:

- throws java.io.IOException
- try – catch()



CLASSE JAVA.UTIL.SCANNER

Nova classe incluída a partir da versão J2SE 5.0, facilita o input de dados

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

JAVA.UTIL.SCANNER

❑ Leitura de System.in

```
Scanner sc = new Scanner(System.in);  
int i = sc.nextInt();
```

❑ Leitura de valores do tipo *long* de um ficheiro

```
Scanner sc = new Scanner(new File("nomeFicheiro"));  
while (sc.hasNextLong()) {  
    long aLong = sc.nextLong();  
}
```

JAVA.UTIL.SCANNER

```
String input = "1 fish 2 fish red fish blue fish";  
Scanner s = new Scanner(input).useDelimiter("\\s*fish\\s*");  
System.out.println(s.nextInt());  
System.out.println(s.nextInt());  
System.out.println(s.next());  
System.out.println(s.next());  
s.close();
```

Atenção!!!

Por defeito o delimitador é o espaço.
No exemplo está a utilizar o delimitador fish

❑ Output:

1

2

red

blue

JAVA.UTIL.SCANNER

- Antes para ler um ficheiro texto

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.io.File;

public class TextReader {
    private static void readFile(String fileName) {
        try {
            File file = new File(fileName);
            FileReader reader = new FileReader(file);
            BufferedReader in = new BufferedReader(reader);
            String string;
            while ((string = in.readLine()) != null) {
                System.out.println(string);
            }
            in.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```
public static void main(String[] args) {
    if (args.length != 1) {
        System.err.println("usage: java TextReader "
            + "file location");
        System.exit(0);
    }
    readFile(args[0]);
}
```


❑ Com a classe Scanner

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class TextScanner {

    private static void readFile(String fileName) {
        try {
            File file = new File(fileName);
            Scanner scanner = new Scanner(file);
            while (scanner.hasNext()) {
                System.out.println(scanner.next());
            }
            scanner.close();
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

```
public static void main(String[] args) {
    if (args.length != 1) {
        System.err.println("usage: java TextScanner1"
            + "file location");
        System.exit(0);
    }
    readFile(args[0]);
}
```

JAVA.UTIL.SCANNER

- ❑ Consultar informação sobre a classe `java.util.Scanner`.

I/O COM SCANNER

Exemplo :

```
int n;
```

```
Scanner input = new Scanner(System.in);
```

```
n = input.nextInt();
```



PRINTF, FOR EACH

Novas funcionalidades incluídas a partir da versão J2SE 5.0
Exemplos de: Java - How to Program, Deitel & Deitel

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

OUTPUT FORMATADO: PRINTF

- ❑ O método ***printf*** foi incluído a partir da versão J2SE 5.0, “*inspirada*” pela instrução existente na linguagem C.

System.out.printf(formatos, argumentos)

- ❑ `System.out.printf(“%d\n”, 31); // 31`
- ❑ `System.out.printf(“%o\n”, 31); // 37`
- ❑ `System.out.printf(“%x\n”, 31); // 1f`
- ❑ `System.out.printf(“%X\n”, 31); // 1F`

OUTPUT FORMATADO: PRINTF

❑ Exemplos com String e caracteres

```
char character = 'A';
```

```
String string = "This is also a string";
```

```
Integer integer = 1234;
```

```
System.out.printf( "%c\n", character );
```

```
System.out.printf( "%s\n", "This is a string" );
```

```
System.out.printf( "%s\n", string );
```

```
System.out.printf( "%S\n", string );
```

```
System.out.printf( "%s\n", integer );
```

EXEMPLOS PRINTF()

```
// get current date and time
Calendar dateTime = Calendar.getInstance();

// printing with conversion characters for date/time compositions
System.out.printf( "%tc\n", dateTime );
System.out.printf( "%tF\n", dateTime );
System.out.printf( "%tD\n", dateTime );
System.out.printf( "%tr\n", dateTime );
System.out.printf( "%tT\n", dateTime );

// printing with conversion characters for date
System.out.printf( "%1$tA, %1$tB %1$td, %1$tY\n", dateTime );
System.out.printf( "%1$TA, %1$TB %1$Td, %1$TY\n", dateTime );
System.out.printf( "%1$ta, %1$tb %1$te, %1$ty\n", dateTime );

// printing with conversion characters for time
System.out.printf( "%1$tH:%1$tM:%1$tS\n", dateTime );
System.out.printf( "%1$tZ %1$tl:%1$tM:%1$tS %1$tp\n", dateTime );
```

EXEMPLOS PRINTF() (OUTPUT)

```
Qua Nov 25 11:04:55 GMT 2009
2009-11-25
11/25/09
11:04:55 AM
11:04:55
Quarta-feira, Novembro 25, 2009
QUARTA-FEIRA, NOVEMBRO 25, 2009
Qua, Nov 25, 09
11:04:55
GMT 11:04:55 am
```


EXEMPLOS PRINTF()

```
Object test = null;  
System.out.printf( "%b\n", false );  
System.out.printf( "%b\n", true );  
System.out.printf( "%b\n", "Test" );  
System.out.printf( "%B\n", test );  
System.out.printf( "Hashcode of \"hello\" is %h\n", "hello" );  
System.out.printf( "Hashcode of \"Hello\" is %h\n", "Hello" );  
System.out.printf( "Hashcode of null is %H\n", test );  
System.out.printf( "Printing a %% in a format string\n" );  
System.out.printf( "Printing a new line %nnext line starts here\n" );
```

EXEMPLOS PRINTF() (OUTPUT)

```
false  
true  
true  
FALSE  
Hashcode of "hello" is 5e918d2  
Hashcode of "Hello" is 42628b2  
Hashcode of null is NULL  
Printing a % in a format string  
Printing a new line  
next line starts here
```

EXEMPLOS PRINTF()

```
public static void main(String args[]) {  
    System.out.printf("%d %(d %+d %05d\n", 3, -3, 3, 3);  
  
    System.out.printf("Default floating-point format: %f\n", 1234567.123);  
    System.out.printf("Floating-point with commas: %,f\n", 1234567.123);  
    System.out.printf("Negative floating-point default: %,f\n", -1234567.123);  
    System.out.printf("Negative floating-point option: %,(f\n", -1234567.123);  
  
    System.out.printf("Line-up positive and negative values:\n");  
    System.out.printf("% ,.2f\n% ,.2f\n", 1234567.123, -1234567.123);  
}
```

```
3 (3) +3 00003  
Default floating-point format: 1234567.123000  
Floating-point with commas: 1,234,567.123000  
Negative floating-point default: -1,234,567.123000  
Negative floating-point option: (1,234,567.123000)  
Line-up positive and negative values:  
 1,234,567.12  
-1,234,567.12
```

Referência: http://www.java2s.com/Tutorial/Java/0120__Development/0200__printf-Method.htm

PRINTF [INDICE_ARGS\$]

- É possível indicar o argumento a que nos estamos a referir na string de formatação:

```
System.out.printf("a1=%1$d a2=%1$d a3=%1$d", x);
```

```
int x = 1234;
```

```
int i1=5,i2=4;
```

```
char c = '\n';
```

```
System.out.printf("i1 = %2$d %1$c i2 = %3$d %1$c ", '\n', 5, 4);
```

```
System.out.printf("i1 = %2$d %1$c i2 = %3$d %1$c ", c, i1, i2, i3);
```

```
System.out.printf("Inteiro = %d Octal = %1$0 Hexadecimal = %1$h \n", x);
```

FOR EACH

- ❑ No Java 5, surge uma nova forma de utilização do ciclo for, para a manipulação e iteração de uma forma mais vantajosa com *arrays* e outros conjuntos de dados.

for (idTipo e : idArray) { *Instruções* }

“Para cada elemento **e** de tipo *idTipo* do array *idArray* fazer ...”

FOR EACH - EXEMPLOS

```
int a[] = {1,2,3,4,5,6,7,8,9,0};  
for (int i : a) System.out.println(""+i);
```

```
double[] ar = {1.2, 3.0, 0.8};  
int soma = 0;  
for (double d : ar) {  
    soma += d;  
}
```

FOR EACH - EXEMPLOS

```
String [][] nomes = {{"Maria", "Pedro"},  
                      {"José", "Ana"},  
                      {"Joana", "Manuel"}};  
  
String s = "";  
    for (String[] lnomes : nomes){  
        for (String nome : lnomes){  
            s += nome;  
        }  
    }  
    System.out.println(""+s);
```

Output: MariaPedroJoséAnaJoanaManuel

FOR EACH - EXEMPLOS

```
Vector t = new Vector(); // Aviso! Vector Obsolete Collection
```

```
t.addElement(new Integer(1));
```

```
t.addElement(new Integer(2));
```

```
t.addElement(new Integer(3));
```

```
t.addElement(new Integer(4));
```

```
t.addElement(new Integer(5));
```

```
for (Object i : t) System.out.println(""+i.toString());
```


FOR EACH - EXEMPLOS

```
Vector t = new Vector();
```

```
t.addElement("a");
```

```
t.addElement("bb");
```

```
t.addElement("ccc");
```

```
t.addElement("dddd");
```

```
t.addElement("eeeeee");
```

```
for (Object i : t) System.out.println(""+i.toString());
```

- ❑ Ler e guardar as notas de n alunos:
 - ❑ Notas entre 0 e 20;
 - ❑ Calcular e escrever a média;
 - ❑ Calcular e escrever a maior e menor nota;
 - ❑ Calcular e escrever a percentagem de aprovados e reprovados.
 - ❑ Visualizar todas as notas ordenadas por ordem decrescente.
 - ❑ Associar a cada nota o respetivo nome.

- ❑ Dias decorridos entre duas datas.
 - ❑ Escreva um programa que permita calcular o número de dias decorridos entre duas datas expressas em dia, mês, ano.

- ❑ Pretende-se simular um processo eleitoral. Para a simulação do programa deve gerar um valor aleatório, o voto, entre 0 e 9, 50000 vezes.

votos = 0	: voto nulo
votos = 1	: voto no concorrente 1
votos = 2	: voto no concorrente 2
...	...
votos = 9	: voto no concorrente 9

- ❑ Visualizar os resultados.
- ❑ Calcular a percentagem obtida pelos diferentes concorrentes.
- ❑ Para os 3 mais votados, ordenados por ordem decrescente, faça um gráfico de percentagens, desenhando um asterisco por cada 5%.



IPG

Politécnico
da Guarda

Escola Superior
de Tecnologia e Gestão

RECURSIVIDADE

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

RECURSIVIDADE

- Quando uma função se chama a ela própria.

```
static int factorial(int valor) {
    if (valor <= 1) {
        return 1;
    } else {
        return b *
factorial(valor - 1);
    }
}

static int fibonacci(int a) {
    if ((a ==1) || (a == 2))
        return 1;
    else
        return (fibonacci(a-1) +
fibonacci(a-2));
}
```

EXERCÍCIOS PRÁTICOS

❑ Recursividade

- ❑ Soma linear, somar os valores inteiros de 1 até n
- ❑ Soma linear, somar os n valores de um vetor
- ❑ Inversão dos elementos de um vetor
- ❑ Multiplicação de dois valores, usando somas
- ❑ Determinar se um valor é primo
- ❑ Soma dos dígitos de um número inteiro



PROGRAMAÇÃO ORIENTADA AOS OBJECTOS

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

OBJECTOS

- ❑ A noção de objecto é crucial para o paradigma da POO, e que pretende concentrar em si as propriedades seguintes:
 - ❑ a independência de contexto;
 - ❑ a abstracção de dados;
 - ❑ o encapsulamento;
 - ❑ a modularidade.

O QUE É UM OBJECTO?

- ❑ Um objecto é uma representação abstracta de um entidade autónoma sob a forma de:
 - ❑ Um identificador único;
 - ❑ Um conjunto de atributos privados, o estado interno do objecto;
 - ❑ Um conjunto de operações que permitem aceder ao estado interno;

O QUE É UM OBJECTO?

- ❑ Este conjunto de operações que permitem aceder ao estado interno, podem ser:
 - ❑ públicas;
 - ❑ privadas.
- ❑ Estas operações representam no seu conjunto o comportamento do objecto.

OBJECTOS: ANALOGIA AO MUNDO REAL

- ❑ Tudo à nossa volta são objectos.
- ❑ Exemplos:
 - ❑ a mesa;
 - ❑ a bicicleta;
 - ❑ o cão;
- ❑ Estes objectos tem duas características fundamentais:
 - ❑ **estado**; o cão, tem nome, pelo, fome,...
 - ❑ **comportamento**; abana a cauda, ladra,...

OBJECTOS: ANALOGIA AO MUNDO REAL

❑ Outro exemplo:

- ❑ A bicicleta tem estado: velocidade corrente, cadência corrente, número de velocidade, duas rodas;
- ❑ A bicicleta tem comportamento: trava, acelera, muda as velocidades

OBJECTO

- ❑ Aos identificadores que guardam os valores dos atributos, ou do estado, de um dado objecto designamos de por ***variáveis***
- ❑ As operações que representam o seu comportamento, ou seja, as computações a realizar internamente, designamos de ***métodos***, representados por subrotinas associadas ao objecto.

OBJECTOS

- ❑ Um único objecto, por si só, não é muito útil.
- ❑ Uma bicicleta é incapaz de qualquer actividade. A bicicleta é útil quando interactua com outro objecto, uma **pessoa**, que por sua vez através dos **pedais** dá movimento à bicicleta.

OBJECTOS : MENSAGENS

- ❑ Os objectos vão interactuar entre si através de um mecanismo de envio de mensagens.
- ❑ Estas mensagens vão ser responsáveis, quer por alterações ao comportamento interno; quer pela determinação de resultados.

CLASSE

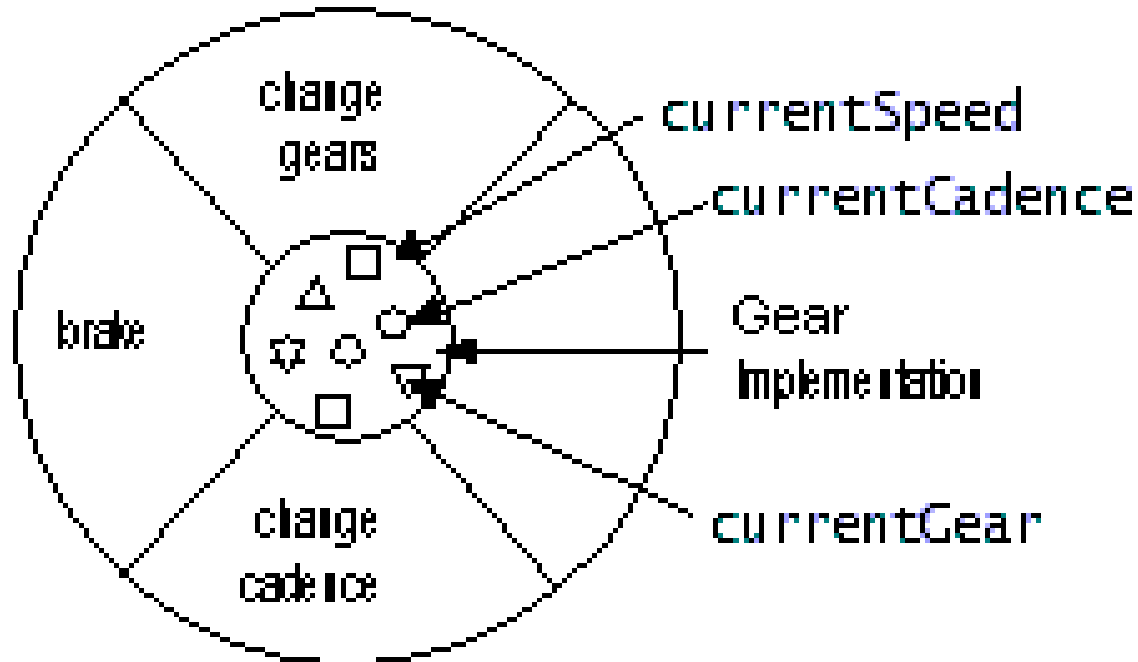
- ❑ No Mundo real, existem muitos objectos do mesmo tipo, que tendo características comuns são reunidos na mesma classe.
- ❑ Por exemplo, classe dos peixes, classe dos mamíferos, a minha bicicleta é apenas uma de um grande conjunto e variedade de bicicletas existente no Mundo.

CLASSE

- ❑ Usando a terminologia OO, dizemos que a minha bicicleta é uma instância da classe bicicletas.
- ❑ As bicicletas têm alguns estados (velocidade corrente, duas rodas) e comportamentos (mudar de velocidade, travar) em comum.
- ❑ Todavia, o estado de cada bicicleta é independente das outras.

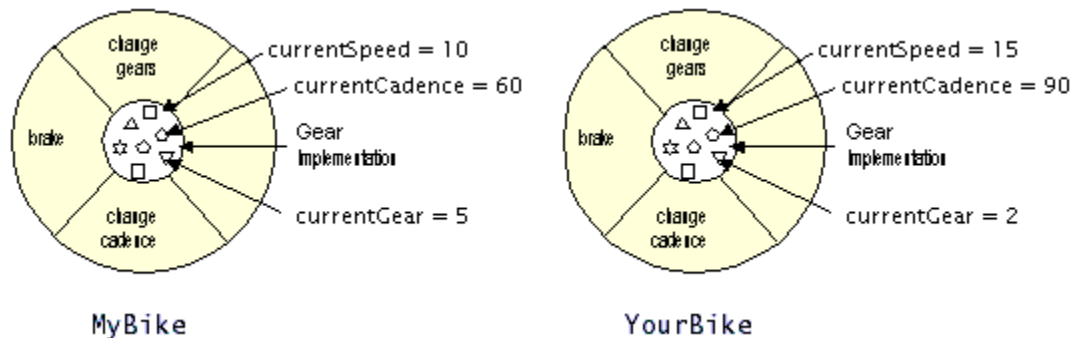
CLASSE: EXEMPLO

□ Classe bicicleta



CLASSE

- Depois de criada a classe bicicleta, podemos criar vários objectos bicicleta a partir da classe.



CLASSE: EXEMPLO

```
class Bicicleta {  
    //  Atributos  
    int currentSpeed;  
    int currentCadence;  
    int currentGear;  
    //  Comportamentos  
    int changeGear();  
    int changeCandece();  
    void brake();  
}
```

CLASSES : HERANÇA

- ❑ Uma classe herda estados e comportamentos da sua superclasse.
- ❑ Por exemplo, bicicletas de montanha, bicicletas de corrida, bicicletas de dois lugares são todas tipos de bicicletas.
- ❑ Em terminologia OO, os tipos de bicicletas descritas são subclasses da classe bicicleta.

CLASSES : HERANÇA

- ❑ Cada subclasse herda o estado e comportamentos da sua superclasse.
- ❑ Cada subclasse não está limitada unicamente aos atributos e comportamentos herdados da sua superclasse.
- ❑ Em cada subclasse podemos adicionar variáveis e métodos.

CLASSES : POLIMORFISMO

- ❑ *Polimorfismo* é um termo utilizado para descrever uma situação em que o mesmo nome pode referir dois métodos diferentes.
- ❑ O polimorfismo pode ser de dois tipos:
 - ❑ *override*
 - ❑ *overload*

CLASSES : POLIMORFISMO

- ❑ *Override*: implementação especial de um método; por exemplo, uma bicicleta de montanha com um tipo especial, ou adicional, de velocidades, teremos a necessidade de implementar (*sobrepôr* = *override*) o método para a mudança de velocidade.

CLASSES : POLIMORFISMO

- ❑ *Overload*: o mesmo nome em métodos com parâmetros diferentes.

(overload = sobrecarregar)

AS CLASSES

❑ Definição

- ❑ É um objecto especial que serve de padrão para a criação de objectos similares por instâncias da classe, objectos que possuem a mesma estrutura interna e interface.
- ❑ Neste sentido uma **CLASSE** é um módulo que especifica quer a estrutura quer o comportamento das suas instâncias.

Exemplo Prático

- ❑ Criar um objecto do tipo *Contador*
- ❑ Requisitos iniciais:
 - ❑ Os contadores deverão ser do tipo inteiro;
 - ❑ Deverá ser possível criar contadores com valor inicial igual a zero;
 - ❑ Deverá ser possível criar contadores com outro valor inicial dado por parâmetro;
 - ❑ Deverá ser possível saber qual o valor actual de um dado contador;
 - ❑ Deverá ser possível incrementar um contador de 1 valor ou com outro valor dado por parâmetro;
 - ❑ Deverá ser possível diminuir um contador de 1 valor ou com outro valor dado por parâmetro;
 - ❑ Deverá ser possível obter uma representação textual de um contador.

CRIAÇÃO DE CLASSES: EXEMPLO

- ❑ A definição de uma classe é em Java realizada dentro de um bloco que é antecedido pela palavra reservada **class**.
- ❑ **class** é seguida do identificador da classe, que deve sempre começar por letra maiúscula.

```
class Contador {  
}
```

CRIAÇÃO DE CLASSES: EXEMPLO

- ❑ Cada contador vai necessitar apenas de uma variável do tipo inteiro para conter o valor da contagem.

```
class Contador {  
    // variáveis de instância  
    int conta;  
}
```

CRIAÇÃO DE CLASSES: EXEMPLO

- ❑ O primeiro código que se define na criação de uma classe, é o de um método particular designado por ***construtor***.
- ❑ Os *construtores* têm por objectivo criar instâncias de uma dada classe.

CRIAÇÃO DE CLASSES: EXEMPLO

- ❑ Em Java, para cada classe definida de nome *X* é automaticamente disponibilizado um construtor *X()*;
- ❑ Para criar instâncias de tal classe usamos a expressão:

new X()

- ❑ Porém, atribuí o valor *null* a todas as variáveis de instância, o que não é nada útil.

CRIAÇÃO DE CLASSES: EXEMPLO

- ❑ Por tal razão, o Java permite a redefinição do *construtor*, dando-lhe uma utilidade e prática muito maior.
- ❑ No nosso exemplo, caso nenhum outro construtor fosse por nós criado, seria:

Contador conta1 = new Contador();

- ❑ Como a variável associada é uma variável inteira o valor inicial seria 0.

CRIAÇÃO DE CLASSES: EXEMPLO

- ❑ O valor 0, é um valor adequado para um objecto que se pretende seja um contador de inteiros.
- ❑ Porém, devemos indicar de forma explícita no nosso construtor o valor de inicialização, para evitar ambiguidades.

//Construtores

```
Contador( ) {  
    conta = 0;  
}
```

CRIAÇÃO DE CLASSES: EXEMPLO

- ❑ Porém, nos nossos requisitos, pretendemos que o nosso contador seja inicializado com um valor expresso num parâmetro.
- ❑ Podemos, então, criar outro construtor:

```
Contador( int valor ) {  
    conta = valor;  
}
```

CRIAÇÃO DE CLASSES: EXEMPLO

- Neste momento, a nossa classe tem a seguinte configuração:

```
class Contador {  
    //Construtores  
    Contador( ) {  
        conta = 0;  
    }  
    Contador( int valor ) {  
        conta = valor;  
    }  
    // variáveis de instância  
    int conta;  
}
```

CRIAÇÃO DE CLASSES

- ❑ Definição do comportamento: início do processo de programação dos métodos
- ❑ Regra crucial para uma correcta programação POO, já abordada:
 - ❑ Nenhuma instância deve aceder directamente às variáveis de outra instância;
 - ❑ Cada instância apenas invoca por mensagens os métodos de acesso a tais valores que devem ser programados em cada cápsula.

CRIAÇÃO DE CLASSES

- ❑ Para a implementação desta regra os autores da linguagem Java, sugerem que todos os métodos que fazem acesso ao valor de uma variável genérica X, devem ser designados por:

getX

- ❑ Todos os métodos que alterem o valor de uma variável genérica X, devem ser designados por

setX

CRIAÇÃO DE CLASSES

❑ A definição de qualquer método é constituída por:

❑ cabeçalho

<tipo de resultado> <identificador> (< parâmetros >)

- corpo do método:

```
{  
    ...  
    instruções;  
    ...  
    return // se devolver valor  
}
```

CRIAÇÃO DE CLASSES: EXEMPLO

- Continuando o nosso exemplo:
 - método para devolver o valor actual do contador

```
int getConta( )  
{  
    return conta;  
}
```


CRIAÇÃO DE CLASSES: EXEMPLO

- ❑ método para incrementar de uma unidade o contador

```
void incConta( )  
{  
    conta++;  
}
```

CRIAÇÃO DE CLASSES: EXEMPLO

- ❑ método que permite incrementar um número de unidades, dado por um parâmetro, ao contador

```
void incConta( int incremento)
{
    conta += incremento;
}
```

CRIAÇÃO DE CLASSES

- ❑ De salientar que na classe ***Contador*** foram declarados dois métodos com o nome ***incConta***, é a isto que chamamos de **sobrecarga de operadores** ou ***overloading***.

CRIAÇÃO DE CLASSES: EXEMPLO

- ❑ Método para diminuir de uma unidade

```
void decConta( )  
{  
    conta --;  
}
```

CRIAÇÃO DE CLASSES: EXEMPLO

- ❑ Método para diminuir de um valor indicado por parâmetro

```
void decConta(int decremento )  
{  
    conta -= decremento;  
}
```

CRIAÇÃO DE CLASSES: EXEMPLO

- ❑ Como representar sobre a forma de uma *string* o valor do contador.
 - ❑ Por exemplo: Contador = 33
- ❑ Para tal, vamos utilizar uma instância particular da classe *String*

```
String toString( )  
{  
    return (new String("Contador = " + this.getConta() ));  
}
```



IPG

Politécnico
da Guarda

Escola Superior
de Tecnologia e Gestão

REFERÊNCIA THIS

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

REFERÊNCIA THIS

- ❑ A referência a *this*
 - ❑ Esta expressão surge na necessidade de invocar o método *getConta()* dentro de um método do mesmo objecto.
 - ❑ O problema que é colocado é: como referenciar o próprio objecto no seu próprio código.
- ❑ O Java, resolve este problema com uma referência especial para cada objecto de nome *this*

REFERÊNCIA THIS

- ❑ *this* é de facto uma forma de auto-referência utilizável só do seu interior.
- ❑ Através desta referência especial *this*, qualquer objecto pode aceder aos seus métodos e até às suas variáveis de referência.

REFERÊNCIA THIS

```
class thisTest
{
    int teste = 5;
    void printMe( )
    {
        int teste = 10;
        System.out.println("Inst = " + this.teste +
            ", Local = " + teste);
    }
}
```

Output:

Inst = 5 , Local = 10

CRIAÇÃO DE CLASSES: THIS

- ❑ No exemplo apresentado, tal problema não se coloca, logo o código poderia simplesmente ficar:

```
String toString( )  
{  
    return (new String("Contador = " + conta ));  
}
```



AGRUPAMENTO DE CLASSES

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

AGRUPAMENTO DE CLASSES

□ Packages

- Em Java as classes são em geral agrupadas em *packages*, em função da sua funcionalidade.
- Qualquer classe ou membro de uma classe pode ser referenciado de forma absoluta usando como prefixo o nome da ***packages***, seguido do identificador que se pretende aceder.

```
java.lang.String.size( );  
java.io.System.out.println(“ Ola “);
```

AGRUPAMENTO DE CLASSES

- ❑ Sempre que necessitamos usar classes pertencentes a um dado *package*, poderemos usar a cláusula de importação existente no Java:
 - ❑ `import java.util.Vector;`
 - ❑ `import java.lang.String;`
- ❑ Importações globais:
 - ❑ `import java.util.*;`
- ❑ Se desejarmos que a nossa classe pertença a uma dada *package*, então na primeira linha do nosso ficheiro fonte colocamos:
 - ❑ `package [nome da classe], ex. package java.minhasclasses`



IPG

Politécnico
da Guarda

Escola Superior
de Tecnologia e Gestão

ACESSIBILIDADE

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

ACESSIBILIDADE DE UMA CLASSE

- ❑ Cada classe pode controlar a informação ou dados que podem ser visíveis por outras classes.
- ❑ Os tipos de acesso possíveis são:
 - ❑ public;
 - ❑ private;
 - ❑ protected;
 - ❑ *definição por omissão;*

ACESSIBILIDADE DE UMA CLASSE

- ❑ Os modificadores de acessibilidade definem se outras classes podem usar um campo ou método específico.
- ❑ Existem dois níveis de controlo de acesso:
 - ❑ Ao nível da classe (top-level): **public** ou *não definido* (ou não explícito, significa que não definido nenhum tipo de acessibilidade, e neste caso, o Java trata como **package-private**).
 - ❑ Ao nível dos membros da classe (member-level): **public**, **private**, **protected**, **package-private**.

ACESSIBILIDADE DE UMA CLASSE

□ Principais regras:

- Se uma classe é declarada como **public**, esta classe passa a ser visível por todas as classes em qualquer local; se declarada como **package-private**, é visível somente dentro do próprio package.
- O quadro, Níveis de acesso, resume todas as outras opções:

Níveis de Acesso				
Modificador	Classe	Package	Subclasse	Mundo
public	Y	Y	Y	Y
protected	Y	Y	Y	N
<i>sem modificador</i>	Y	Y	N	N
private	Y	N	N	N

EXEMPLO: CLASS CONTADOR

- ❑ Se desejarmos que a nossa classe faça parte do package: *java.minhasclasses*

```
package java.minhasclasses
```

```
class Contador { ...
```

- ❑ Para tornar a classe pública, ou seja, acessível por todos:

```
package java.minhasclasses
```

```
public class Contador { ...
```

ACESSIBILIDADE A VARIÁVEIS E MÉTODOS

- ❑ Por razões de preservação do encapsulamento, é boa prática não declarar variáveis por omissão de acesso nem como sendo públicas, e criar métodos de acesso específicos que são declarados como *public*;
- ❑ Um método *public* é acessível de qualquer ponto de um programa Java;
- ❑ A API de uma classe Java é o conjunto dos métodos que não são declarados como *private*;

ACESSIBILIDADE A VARIÁVEIS E MÉTODOS

- ❑ Um método sem modificador de acesso é acessível a qualquer classe do mesmo *package*;
- ❑ Um método ou variável declarados como *private* são apenas acessíveis dentro da própria classe;
- ❑ Um método ou variável declarados como *protected* são acessíveis na própria classe, de outra classe do mesmo *package* e nas subclasses da classe.

- ❑ Exemplo: Projecto: ExAcessibilidade

REESCREVENDO O EXEMPLO

```
public class Contador {  
    //Construtores  
    public Contador() {  
        conta = 0;  
    }  
    public Contador( int valor ) {  
        conta = valor;  
    }  
    // variáveis de instância  
    private int conta;  
  
    public int getConta() {  
        return conta;  
    }  
}
```

```
public void incConta() {  
    conta++;  
}  
public void incConta( int incremento ) {  
    conta += incremento;  
}  
public void decConta() {  
    conta --;  
}  
public void decConta(int decremento ) {  
    conta -= decremento;  
}  
public String escreveString() {  
    return (new String("Contador = "  
        + conta );  
}
```

TESTE DA CLASSE CONTADOR

```
public class TesteContador {  
  
    // Classe de teste da Classe Contador  
    // método único com todo o código de teste  
  
    public static void main(String args[]) {  
  
        // Código de teste  
  
    }  
  
}
```

TESTE DA CLASSE CONTADOR

```
public class TesteContador {  
  
    // Classe de teste da Classe Contador  
    // método único com todo o código de teste  
  
    public static void main(String args[]) {  
  
        // Criação de Instâncias  
        Contador ct1, ct2, ct3;  
        ct1 = new Contador();  
        ct2 = new Contador(20);  
        ct3 = new Contador(10);  
  
        // Utilização das Instâncias  
  
        int c1, c2, c3;          // variáveis auxiliares  
        c1 = ct1.getConta();  
        c2 = ct2.getConta();  
    }  
}
```


TESTE DA CLASSE CONTADOR

// primeira saída de resultados para verificação

```
System.out.println("c1 = " + c1);
```

```
System.out.println("c2 = " + c2);
```

// alterações às instâncias e novos resultados

```
ct1.incConta(); ct2.incConta(12);
```

```
c1 = ct1.getConta(); c2 = ct2.getConta();
```

```
c3 = c1 + c2; System.out.println("c1 + c2 = " + c3);
```

```
ct3.decConta(); ct2.decConta(5);
```

```
c1 = ct2.getConta(); c2 = ct3.getConta();
```

```
c3 = c1 + c2; System.out.println("c1 + c2 = " + c3);
```

// conversão para string e apresentação

```
System.out.println(ct1.escreveString());
```

```
System.out.println(ct2.escreveString());
```

```
}
```

```
}
```

CLASSES USANDO COMPOSIÇÃO

- ❑ Em Java, como em qualquer linguagem de POO, existe um mecanismo básico e simples de uma classe poder usar na sua definição classes já definidas.
- ❑ Este mecanismo designa-se por **Composição**
- ❑ A composição vem favorecer a reutilização

CLASSES USANDO COMPOSIÇÃO

- Um dos exemplos típicos da POO é a criação de uma classe que represente contas bancárias.
- A classe ***ContaBanco***, com as características seguintes:
 - Uma conta bancária pode ter 1 ou mais titulares, sendo um deles o titular principal, do qual se conhece a morada;

CLASSES USANDO COMPOSIÇÃO

- Cada conta possui um saldo actual e um *plafond* de crédito que pode variar de conta para conta, mas que é definido quando esta é criada;
- A qualquer altura é possível realizar um depósito;
- Um levantamento apenas pode ser realizado se a importância pedida não ultrapassar o *plafond* de crédito definido;
- A qualquer momento deverá ser possível saber o saldo da conta;
- Deverá ser possível eliminar titulares e acrescentar titulares novos;
- Deverá registar-se o número total de movimentos activos realizados sobre a conta, ou seja depósitos e levantamentos;

EXEMPLO: CLASS CONTABANCO

```
public class ContaBanco {  
    // Variáveis de instância  
    String numConta;  
    String morada;  
    Vector titulares;  
    int saldo;  
    int numMov;  
    int plafond;  
}
```

EXEMPLO: CLASS CONTABANCO

//Métodos de acesso às variáveis

String getTitular() { ... }

int getSaldo() { ... }

int getNumMov() { ... }

int getPlafond() { ... }

Object[] getTitulares() { ... }

EXEMPLO: CLASS CONTABANCO

//Métodos modificadores do estado interno das variáveis

void deposita(int valor)

void levanta(int valor)

void insereTit(String titular)

void alteraMorada(String mora)

void alteraPlafond(int nplaf)

EXEMPLO: CLASS CONTABANCO

//Construtor

```
public ContaBanco(String titp, String mora,
    int sld, int plf) {
    titulares = new Vector(5);
    titulares.insertElementAt(titp,0);
    morada = mora;
    if (sld >=0) saldo=sld; // Valida saldo
    else saldo=0;
    plafond = (plf >= 0 ? plf : 0); // Valida plafond
    numMov = 0;
}
```


EXEMPLO: CLASS CONTABANCO

```
// Métodos interrogadores de estado
    public String getNumConta() {
        return numConta;
    }
    public String getTitular() {
        return titulares.firstElement();
    }
    public getSaldo() {
        return saldo;
    }
```

EXEMPLO: CLASS CONTABANCO

```
// Métodos interrogadores de estado
    public int getNumMov() {
        return numMov;
    }
    public int getPlafond() {
        return plafond;
    }
    public Object[] getTitulares() {
        return titulares.toArray();
    }
}
```

EXEMPLO: CLASS CONTABANCO

```
// Métodos modificadores de estado
public void deposita(int valor) {
    saldo += valor;
    numMov ++;
}
public void insereTit(String titular) {
    titulares.addElement(titular);
}
public void alteraMorada(String mora) {
    morada = mora;
}
```

EXEMPLO: CLASS CONTABANCO

```
// Métodos modificadores de estado
    public void alteraPlafond(int nplaf) {
        plafond = nplaf;
    }
    public boolean preLevanta(int valor) {
        return (saldo + plafond) >= valor;
    }
    public void levanta(int valor) {
        saldo -= valor;
        numMov --;
    }
```

COMPLEMENTO NA DEFINIÇÃO DE CLASSE

- As classes são objectos particulares dado que guardam a estrutura e o comportamento que vai ser comum a todas as instâncias a partir de si criadas.

COMPLEMENTO NA DEFINIÇÃO DE CLASSE

- ❑ Às variáveis que representam a estrutura interna de uma dada classe designamos por variáveis de classe.
- ❑ Aos métodos que implementam o comportamento de uma classe designamos por métodos de classe.
- ❑ Manter o mesmo princípio do encapsulamento.

COMPLEMENTO NA DEFINIÇÃO DE CLASSE

- ❑ Relativamente, ao exemplo apresentado, a definição da classe *ContaBanco*, poderemos ter o seguinte:
 - ❑ Uma variável que guarde o número de contas já criadas;
 - ❑ Uma variável que guarde o somatório dos saldos das conta existentes.
- ❑ Estas variáveis seriam ***variáveis de classe***.

COMPLEMENTO NA DEFINIÇÃO DE CLASSE

- A estrutura de especificação de uma classe, será:

Classe

variáveis da classe

métodos da classe

construtor

variáveis de instância

métodos de instância

COMPLEMENTO NA DEFINIÇÃO DE CLASSE

- ❑ As declarações de classe são idênticas às declarações já estudadas para os métodos e variáveis de instância, excepto na utilização da palavra reservada ***static***.
- ❑ ***Static*** indica que existe apenas uma cópia de tal variável que é associada à classe.

COMPLEMENTO NA DEFINIÇÃO DE CLASSE

```
public class ContaBanco {
```

```
//Variáveis de classe
```

```
public static int numContas = 0;
```

```
public static int saldoTotal = 0;
```

COMPLEMENTO NA DEFINIÇÃO DE CLASSE

```
//Métodos de classe  
public static int getNumContas( ) {  
    return numContas;  
}  
public static int getSaldoTotal( ) {  
    return saldoTotal;  
}  
public static void incNumContas( ) {  
    numContas++;  
}  
public static void actSaldoTotal( ) {  
    saldoTotal +=saldo;  
}
```

EXEMPLO: CONTABANCO

- ❑ Implementar classe de teste para simular o pretendido.

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

- ❑ Pretende-se construir uma classe cuja estrutura e comportamento programado simule uma Máquina Genérica de Venda de produtos de dado tipo (ex.chocolates, tabaco, bebidas).

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

- ❑ A Classe *MaqVenda* deverá definir a seguinte informação:
 - ❑ tipo de produto genérico que vende (chocolates, bebidas, tabaco, etc.)
 - ❑ tabela de triplos produto-preço-quantidade existentes na máquina;
 - ❑ total de dinheiro existente na máquina;
 - ❑ número total de compras realizadas;
 - ❑ estado da máquina (avariada, sem dinheiro, sem produtos, ok)

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

- ❑ E ainda, o seguinte comportamento:
 - ❑ criar uma nova máquina de vendas de dado produto;
 - ❑ determinar que tipo de produto vende a máquina;
 - ❑ determinar para uma dada máquina o preço de dado produto;
 - ❑ determinar o estado de uma máquina;
 - ❑ determinar o número total de máquinas criadas;
 - ❑ realizar uma compra válida de dado produto;
 - ❑ alterar o estado de uma dada máquina;
 - ❑ carregar uma máquina com dinheiro ou com dado produto.

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

- ❑ A classe *TriploMaq*, implementa os triplos representativos da informação de um produto de uma máquina de venda:
 - ❑ Nome do produto;
 - ❑ Preço do produto;
 - ❑ Quantidade existente na máquina do produto.

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

❑ Implementação da classe *TriploMaq*:

```
public class TriploMaq {  
  
    public TriploMaq() {  
        produto = new String("");  
        preco = 0;  
        quantidade = 0;  
    }  
  
    public TriploMaq(String nomeprod, int precox,int quantidadex) {  
        produto = new String(nomeprod);  
        preco = precox;  
        quantidade = quantidadex;  
    }  
}
```

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

❑ Implementação da classe *TriploMaq*(continuação):

```
private String produto;  
private int preco;  
private int quantidade;  
  
public String getProduto(){  
    return produto;  
}  
public int getPreco(){  
    return preco;  
}  
public int getQuantidade(){  
    return quantidade;  
}
```

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

- ❑ Implementação da classe *TriplaMaq*(continuação):

```
public String toString(){  
    return (new String("Nome do Produto: "+produto+" Preço = " + preco+  
        " Quantidade = " + quantidade));  
}  
  
}
```

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

❑ Implementação da classe *MaqVenda*:

```
public class MaqVenda {  
  
    // variaveis da classe  
    static int numMaquinas;          //Total de maquinas  
    static final int MAXPRODS = 100; //Máximo de produtos  
  
    // metodos da classe  
    public static void incNumMaquinas(){  
        numMaquinas++;  
    }  
    public static int maxProdutos(){  
        return MAXPRODS;  
    }  
}
```

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

❑ Implementação da classe *MaqVenda* (continuação):

```
public MaqVenda(String tproduto, TriploMaq tabinic[],
                int cash, int nprodutos) {

    this.incNumMaquinas();
    tipoproduto = tproduto;
    totalDinheiro = cash;
    tabProdutos = tabinic;
    numProdutos = (nprodutos <= this.maxProdutos) ?
                                                           nprodutos : this.maxProdutos();

    numCompras = 0;
    estado = new String("OK");

}
```

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

❑ Implementação da classe *MaqVenda* (continuação):

```
// variaveis de instancia
private String tipoproduto;
private int totalDinheiro;
private int numCompras;
private String estado;
private TriploMaq[] tabProdutos;
private int numProdutos;

// métodos de instancia privados
private TriploMaq daTriplo(int index){
    TriploMaq triplo = (TriploMaq) tabProdutos[index].clone();
    return triplo;
}
```

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

❑ Implementação da classe *MaqVenda* (continuação):

```
// Procura nome do produto, devolve -1 se o nome não existir
private int procuraProduto(String produto){
    boolean enc = false;
    int ind = -1;
    while(!enc && (ind < numProdutos)) {
        ind++;
        enc = ((this.daTriplo(ind).getProduto()).equals(produto) ? true : false);
    }
    return (enc ? ind : -1);
}
// metodos de instancia
public String getProduto() {
    return tipoproduto;
}
```

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

❑ Implementação da classe *MaqVenda* (continuação):

```
public int getDinheiro() {  
    return totalDinheiro;  
}  
  
public int getNumCompras() {  
    return numCompras;  
}  
public int getNumProdutos() {  
    return numProdutos;  
}  
  
public String getEstado(){  
    return estado;  
}
```


EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

❑ Implementação da classe *MaqVenda* (continuação):

```
// Devolve uma copia do array de triplos
public TriploMaq[] daTab() {
    TriploMaq t[] = new TriploMaq[MaqVenda.maxProdutos()];
    System.arraycopy(tabProdutos,0,t,0,numProdutos);
    return t;
}

public int precoDe(String prod){
    int ind = this.procuraProduto(prod);
    return (ind != -1 ? this.daTriplo(ind).getPreco() : 0);
}
```

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

❑ Implementação da classe *MaqVenda* (continuação):

```
public void compra(String prod, int quantia) {
    int ind = this.procuraProduto(prod);
    totalDinheiro += tabProdutos[ind].getPreco();
    numCompras++;
    tabProdutos[ind] = new TriploMaq(prod, daTriplo(ind).getPreco(),
                                     daTriplo(ind).getQuantidade()-1);
}
//Pre-condicao de compra; produto existe;
//                                quantidade>0; maquina "OK"
public boolean preCompra(String prod) {
    int i = this.procuraProduto(prod);
    return( (i!=-1) && (tabProdutos[i].getQuantidade())>0
           && (this.getEstado().equals("OK")));
}
```

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

❑ Implementação da classe *MaqVenda* (continuação):

```
public void mudaEstado(String novoestado){
    estado = novoestado;
}

public boolean estadoOk(String est){
    return (est.equals("OK") || est.equals("SP") || est.equals("SD") ||
            est.equals("AV"));
}

//Carregar a maquina com mais dinheiro
public void maisDinheiro(int dinheiro){
    totalDinheiro += dinheiro;
}
```

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

❑ Implementação da classe *MaqVenda* (continuação):

```
//insere produto
public void insNovoProduto(String prod, int preco, int qt) {
    tabProdutos[this.getNumProdutos()] = new TriploMaq(prod,preco,qt);
    this.numProdutos++;
}
//teste para inserir novo produto
public boolean preInsNovoProduto(String prod, int preco,int qt){
    return (this.getNumProdutos() < 20 &&
            this.procuraProduto(prod)==0 && preco>0 && qt>0);
}

}
```

EXEMPLO: MÁQUINA AUTOMÁTICA DE VENDAS

- ❑ Implementar classe de teste para simulação do pretendido.



CLASSES NÃO INSTANCIÁVEIS

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

CLASSES NÃO INSTANCIÁVEIS

- ❑ Partindo de uma classe, é possível criar tantas instâncias idênticas em estrutura e comportamento quantas as necessárias.
- ❑ Existem, no entanto, circunstâncias especiais em que não faz sentido a criação de quaisquer instâncias.

CLASSES NÃO INSTANCIÁVEIS

- ❑ Se uma dada classe for definida como tendo apenas variáveis e métodos de classe, então essa classe não especifica a estrutura e o comportamento de qualquer instância, pelo que estas não poderão ser criadas, nem tal faria sentido.

CLASSES NÃO INSTANCIÁVEIS

- ❑ Este tipo de classes não instanciáveis, são auxiliares preciosos em POO, representando centros de serviços.
- ❑ Por exemplo, a classe *Math* que pertence ao *package java.lang*, coloca ao serviço dos utilizadores um conjunto de serviços matemáticos

CLASSES NÃO INSTANCIÁVEIS

❑ Utilização da classe *Math*:

- ❑ `double x = Math.sqrt(y);` // raiz quadrada
- ❑ `double x = Math.cos(y);` // coseno
- ❑ `long i = Math.round(y);` // arredondamento
- ❑ `double x = Math.max(y,z);` // Maior de y e z



EXEMPLO: FACTORIAL

- ❑ Como foi apresentado, possivelmente a melhor forma para o calculo do Factorial, seria:

```
public class MathX {  
    public static int factorial(int n) {  
        int i, total=1;  
        for(i=n;i>1;i--) total *= i;  
        return total; }  
}
```

```
public class TesteNovoFactorial {  
  
    public static void main (String args[]) {  
        System.out.println(MathX.Factorial(5));}  
}
```



HIERARQUIA DE CLASSES E HERANÇA

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

Bibliografia:

Martins, F. Mário , "*Programação Orientada aos Objectos em Java2*",

FCA,ISBN: 972-722-196-3

Eckel, Bruce, "*Thinking em Java*"

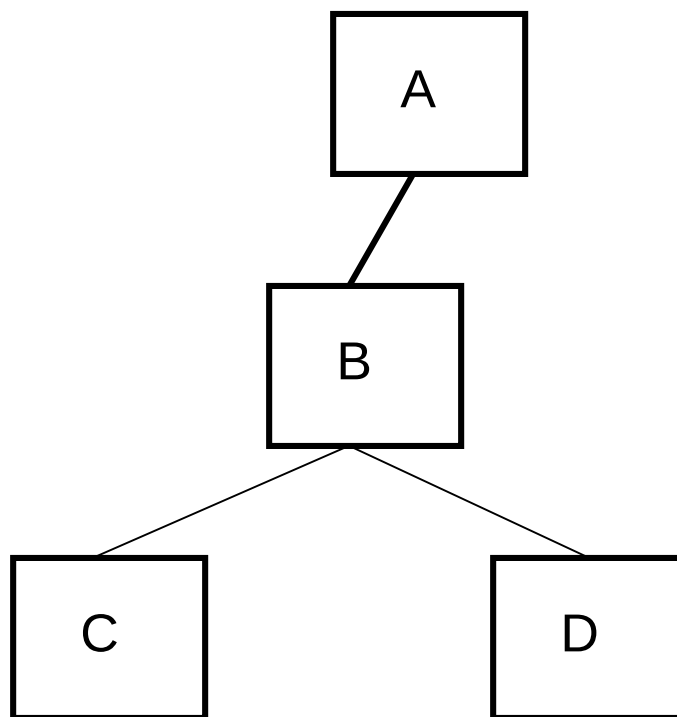
Prentice-Hall, Junho, 2000,

ISBN: 0-13-027363-5

HIERARQUIA DE CLASSES E HERANÇA

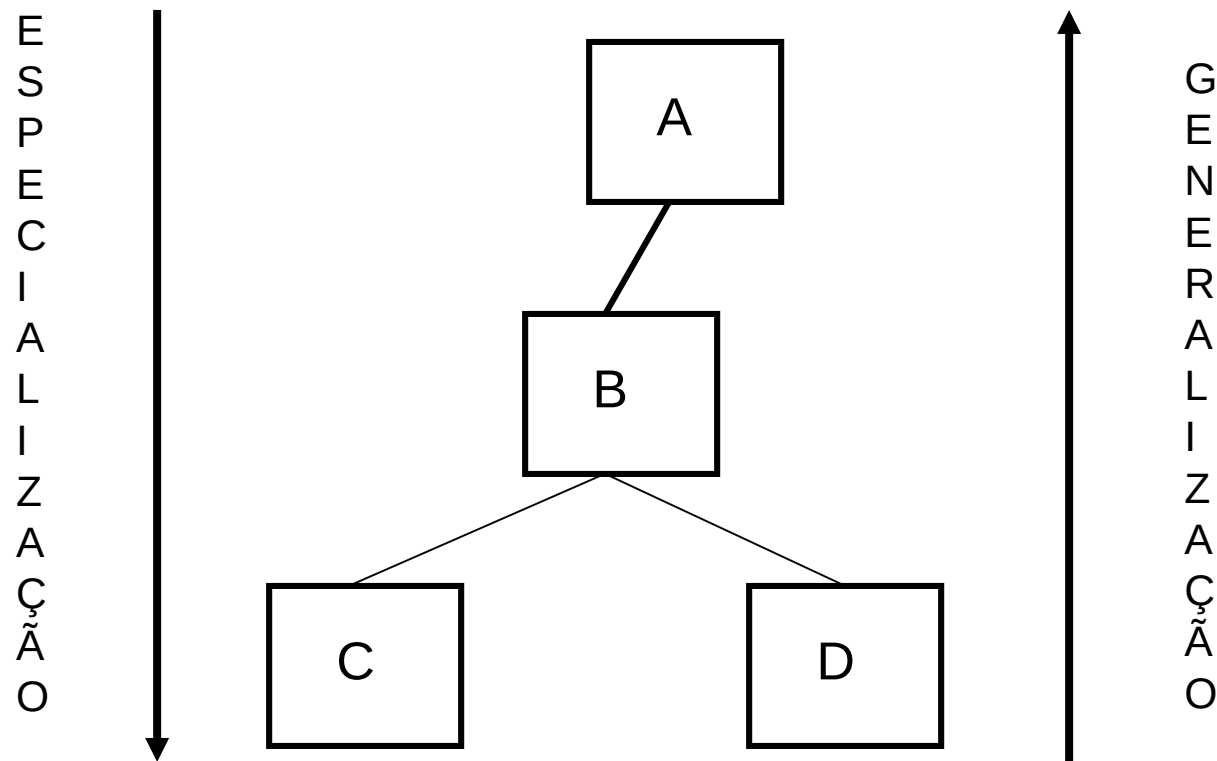
- ❑ Torna-se muito importante criar um mecanismo que facilite a definição de classes à custa de classes existentes.
- ❑ Este mecanismo deve ser baseado nas noções de similaridade e especialização entre classes.
- ❑ Em POO, a solução para este problema consistiu em introduzir um espaço para a definição de classes hierárquico.

HIERARQUIA DE CLASSES



Noção de:
- Superclasse;
- Subclasse

HIERARQUIA DE CLASSES



O MECANISMO DE HERANÇA

- ❑ Em POO, entre as classes de uma hierarquia é estabelecida um relação de especialização, que é automaticamente implementada através de um mecanismo de herança.

O MECANISMO DE HERANÇA

- ❑ Este mecanismo automático de herança estabelece as seguintes propriedades entre uma subclasse B e a sua superclasse A:
 - ❑ B herda de A todas as variáveis e métodos que não sejam declarados como *private*;
 - ❑ B pode definir novas variáveis e novos métodos próprios;
 - ❑ B pode redefinir variáveis e métodos herdados;
 - ❑ a herança não se aplica a variáveis e métodos de classe.

O MECANISMO DE HERANÇA

- ❑ As subclasses são noutros contextos também designadas por **classes derivadas**.
- ❑ Herdar significa em POO ter acesso a, poder invocar, em suma, poder usar o que foi definido na superclasse.

O MECANISMO DE HERANÇA

- Em Java, declarar que uma classe B é subclasse de uma classe A consiste em introduzir no cabeçalho da definição da subclasse:

```
public class B extends A {
```

❑ Referência *super*:

- ❑ A utilização da referência *super*, indica que o código do método deve ser directamente procurado na superclasse do receptor.

O MECANISMO DE HERANÇA

□ Analisemos o exemplo seguinte:

```
public class Um {  
    public Um() { }  
    public int teste(){ return 1; }  
    int result1(){ return this.teste();}  
}
```

```
public class Tres extends Dois {  
    public Tres() { }  
    public int result2(){ return 3; }  
    public int result3(){ return super.teste();}  
}
```

```
public class Dois extends Um {  
    public Dois() { }  
    public int teste(){return 2; }  
}
```

```
public class Quatro extends Tres{  
    public Quatro() { }  
    public int teste(){ return 4;}  
    public int result4(){return super.teste(); }  
}
```

PROCURA DINÂMICA DE MÉTODOS

❑ *Dynamic Method LookUp*

- ❑ O interpretador de Java vai em primeiro lugar pesquisar o método na classe para a qual é dirigida a mensagem. Se o método não for encontrado a procura continua seguindo o algoritmo de ***procura dinâmica de métodos*** nas suas superclasses.

O MECANISMO DE HERANÇA

```
public class TesteHeranca {  
    public static void main(String[] args) {  
  
        Um i1 = new Um();  
        Dois i2 = new Dois();  
        Tres i3 = new Tres();  
        Quatro i4 = new Quatro();  
  
        System.out.println(i4.result2());    // Output 3  
        System.out.println(i1.result1());    // Output 1  
        System.out.println(i4.result1());    // Output 4  
        System.out.println(i4.result4());    // Output 2  
    }  
}
```

CLASSES SEM SUBCLASSES

- ❑ Em Java, é possível definir classes de forma que estas nunca possam ter subclasses.
- ❑ Para que tal propriedade particular seja associada a uma dada classe, basta que no seu cabeçalho seja declarada a palavra ***final***.

```
public final class X {
```


CLASSES SEM SUBCLASSES

- ❑ **Membros do tipo final:**
 - ❑ É uma forma de prevenir a reescrita, overriding, de membros numa subclasse.
 - ❑ Todos os métodos e variáveis podem ser reescritos numa subclasse. Para prevenir essa situação devemos utilizar o qualificador **final**:
 - ❑ Exemplo:

```
final int total = 180;  
final void visualizar();
```

CLASSES SEM SUBCLASSES

- ❑ **Final** assegura que uma funcionalidade definida não pode ser alterada por uma subclasse
- ❑ De modo semelhante se processa quando qualificamos uma classe com **final**.
- ❑ Ou seja, impedimos que a propriedade de herança se concretize. Qualquer tentativa de declarar uma subclasse de uma classe final causará um erro.

UTILIZAÇÃO DE *THIS()* E *SUPER()* EM CONSTRUTORES

- ❑ Em Java, é possível que um construtor de uma classe invoque o construtor de outra classe, usando como código de referência *this()*, com os parâmetros adequados, em tipo e em ordem, à invocação do outro construtor.

UTILIZAÇÃO DE *THIS()* E *SUPER()* EM CONSTRUTORES

- ❑ Quando criamos uma classe B que é subclasse de A, sabemos que herda as variáveis de instância de A. Assim, cada instância criada de B possui, para além das suas variáveis, as variáveis herdadas de A.
- ❑ A inicialização das variáveis de A é efectuada pelos construtores de A, portanto parece lógico que o construtor de B deve invocar obrigatoriamente o construtor de A. A instrução ***super()*** garante a invocação do construtor da superclasse.

O TOPO DA HIERARQUIA: A CLASSE OBJECT

- ❑ A classe *Object* ocupa o topo da hierarquia.
- ❑ A classe *Object* representa o comportamento geral de todas as subclasses enquanto objetos.
- ❑ Na classe *Object* existem um conjunto de métodos muito genéricos, dos quais se salientam:
 - ❑ `Class getClass();` *Devolve a class que representa o objeto.*
 - ❑ `boolean equals(Object obj);` *Compara se os objetos são iguais.*
 - ❑ `Object clone();` *Cria uma cópia do objeto da mesma class.*
 - ❑ `String toString();` *Devolve uma representação textual do objeto.*

CRIAÇÃO DE CLASSES VIA HERANÇA

❑ Exemplo: Pontos2D e Pontos3D

```
public class Pontos2D {  
    //Construtores  
    public Pontos2D(int x, int y) {  
        this.x=x; this.y=y; }  
  
    public Pontos2D() {  
        this(0,0); }  
  
    // Variaveis de Instancia  
    private int x;  
    private int y;  
  
    //Metodos de instancia  
    public int getX(){ return x; }  
    public int getY(){ return y; }  
    public void incX(){ x++; }  
    public void incX(int dx){ x+=dx; }
```

```
    public void incY(){ y++; }  
    public void incY(int dy){ y+=dy; }  
  
    public String toString(){  
        return(new String("Pontos2D("+this.getX()+","  
            +this.getY()+")"));  
    }  
    public void somaPonto(Pontos2D p){  
        this.x += p.getX();  
        this.y += p.getY();  
    }  
    public void somaPonto(int x, int y){  
        this.x += x;  
        this.y += y;  
    }  
}
```

CRIAÇÃO DE CLASSES VIA HERANÇA

❑ Exemplo: Pontos2D e Pontos3D

```
public class Pontos3D extends Pontos2D {
```

```
    public Pontos3D(int x,int y, int z) {
```

```
        super(x,y);
```

```
        this.z = z;
```

```
    }
```

```
    public Pontos3D() {
```

```
        super();
```

```
        this.z = 0;
```

```
    }
```

```
    //variavel de instancia
```

```
    private int z;
```

```
    //Metodos de instancia
```

```
    public int getZ(){
```

```
        return z;
```

```
    }
```

```
    public void somaPonto(int x, int y,int z){
```

```
        super.somaPonto(x,y);
```

```
        this.z += z;
```

```
    }
```

```
    public String toString(){
```

```
        return(new String("Pontos3D("+this.getX()+
```

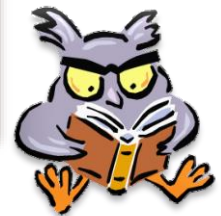
```
            ","+this.getY()+","+this.getZ()+")");
```

```
    }
```

```
}
```

- ❑ Analisar `java.util.Vector`

Actualização
`java.util.ArrayList`



CRIAÇÃO DE CLASSES VIA HERANÇA

❑ Stack como subclasse de Vector

```
import java.util.Vector;
public class Stack extends Vector {

    public Stack() {
        super();
    }
    public boolean empty(){
        return super.isEmpty();
    }
    public void push(Object obj){
        super.addElement(obj);
    }
}
```

```
public Object top(){
    int index = super.size()-1;
    return super.elementAt(index);
}
public void pop(){
    super.removeElementAt(super.size()-1);
}
}
```

Atenção!!! Existe a classe
java.util.Stack

PROGRAMAÇÃO INCREMENTAL

- ❑ O mecanismo de herança introduz uma nova metodologia de programação que se pode designar por **programação incremental**.

Herdado + Programado = Pretendido



VECTOR

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

VECTOR VS ARRAYLIST

- ❑ ArrayList e Vector são bastante idênticos na sua implementação.
- ❑ A principal diferença é a existência de sincronização pela classe Vector.
- ❑ Vários autores defendem que não devemos utilizar os Vectores, pelo facto de a sincronização “gastar” algum tempo e na maioria dos casos a sincronização não ser necessária.

SINCRONIZAÇÃO

- ❑ Métodos sincronizados permitem a implementação de uma estratégia simples de prevenção de erros de interferência e consistência de memória.
- ❑ Ou seja, sincronização permite o controlo de leitura e escrita de diferentes threads à mesma variável.

VECTOR

- ❑ Depois da versão Java SE5.0, surgem novas implementações (Java Framework Collections) e sugere-se a substituição da classe Vector por outra, por exemplo ArrayList.
- ❑ ArrayList é semelhante à classe Vector mas sem sincronização... Mas podemos, a qualquer momento, implementar sincronização na classe ArrayList.

```
ArrayList arrayList = new ArrayList();  
List synchList = Collections.synchronizedList(arrayList);
```



ARRAYLIST

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

ARRAYLIST VS VECTOR

❑ Teste entre ArrayList e Vector. Exemplo de utilização.

```
import java.util.Iterator;

public class Tester {
    public static void main(String[] args) {
        java.util.Random random = new java.util.Random();
        long beginTime;
        long endTime;
        beginTime = System.currentTimeMillis();
        java.util.ArrayList arrayList = new java.util.ArrayList();
        for (int i = 0; i < 1 << 20; i++) {
            arrayList.add(new Integer(random.nextInt()));
        }
        Iterator iterator = arrayList.iterator();
        for (;iterator.hasNext();){
            System.out.println(""+iterator.next());
        }
        java.util.Collections.sort(arrayList);
        endTime = System.currentTimeMillis();
        long totalTimeArrayList = endTime - beginTime;
    }
}
```


ARRAYLIST VS VECTOR

- ❑ Teste entre ArrayList e Vector. Exemplo de utilização.

```
// Teste Vector

beginTime = System.currentTimeMillis();
java.util.Vector vector = new java.util.Vector();
for (int i = 0; i < 1 << 20; i++) {
    vector.add(new Integer(random.nextInt()));
}
java.util.Collections.sort(vector);
endTime = System.currentTimeMillis();
long totalTimeVector = endTime - beginTime;

System.out.println(totalTimeArrayList); //3219
System.out.println(totalTimeVector); //3765
}
}
```

ARRAYLIST

- ❑ ArrayList é redimensionável e implementa List Interface (implementa todas as operações definidas em List).
- ❑ Internamente ArrayList utiliza um array para guardar os elementos.
- ❑ ArrayList contém um conjunto de métodos adicionais para a manipulação dos seus elementos.

ARRAYLIST

- ❑ O tamanho da ArrayList é o número de elementos actual. Sempre que é adicionado um elemento no array, o array é realocado.

ARRAYLIST <E>

```
ArrayList<Aluno> tabela = new ArrayList<Aluno>();  
Iterator<Aluno> it = tabela.iterator();
```

```
tabela.add(new Aluno(122, "Maria"));  
tabela.add(new Aluno(123, "José"));  
tabela.add(new Aluno(124, "Ana"));
```

```
// For each
```

```
for (Aluno s : tabela) {  
    System.out.println(s.toString());  
}
```

```
// For loop with index. This is fast, but should not be used with a LinkedList.
```

```
// It does allow orders other than sequentially forward.
```

```
for (int i = 0; i < tabela.size(); i++) {  
    System.out.println(tabela.get(i).toString());  
}
```

```
// Iterator<E> - Allows simple forward traversal. Can be used for the largest number of other  
// kinds of data structures.
```

```
for (it = tabela.iterator(); it.hasNext();) {  
    System.out.println(it.next().toString());  
}
```

CLASS ARRAYS

- ❑ A class Arrays inclui um conjunto de métodos para manipulação de arrays.

Exemplos do Livro: “Java - How to Program”, Deitel

```
import java.util.Arrays;

public class UsingArrays
{
    private int intArray[] = { 1, 2, 3, 4, 5, 6 };
    private double doubleArray[] = { 8.4, 9.3, 0.2, 7.9, 3.4 };
    private int filledIntArray[], intArrayCopy[];
    ...
}
```

CLASS ARRAYS

...

```
// constructor initializes arrays
public UsingArrays()
{
    filledIntArray = new int[ 10 ]; // create int array with 10 elements
    intArrayCopy = new int[ intArray.length ];

    Arrays.fill( filledIntArray, 7 ); // fill with 7s
    Arrays.sort( doubleArray ); // sort doubleArray ascending

    // copy array intArray into array intArrayCopy
    System.arraycopy( intArray, 0, intArrayCopy,
        0, intArray.length );
} // end UsingArrays constructor
```

...

CLASS ARRAYS

```
...  
// output values in each array  
public void printArrays()  
{  
    System.out.print( "doubleArray: " );  
    for ( double doubleValue : doubleArray )  
        System.out.printf( "%.1f ", doubleValue );  
  
    System.out.print( "\nintArray: " );  
    for ( int intValue : intArray )  
        System.out.printf( "%d ", intValue );  
  
    System.out.print( "\nfilledIntArray: " );  
    for ( int intValue : filledIntArray )  
        System.out.printf( "%d ", intValue );  
  
    System.out.print( "\nintArrayCopy: " );  
    for ( int intValue : intArrayCopy )  
        System.out.printf( "%d ", intValue );  
  
    System.out.println( "\n" );  
} // end method printArrays  
...
```

CLASS ARRAYS

```
...  
// find value in array intArray  
public int searchForInt( int value )  
{  
    return Arrays.binarySearch( intArray, value );  
} // end method searchForInt  
  
// compare array contents  
public void printEquality()  
{  
    boolean b = Arrays.equals( intArray, intArrayCopy );  
    System.out.printf( "intArray %s intArrayCopy\n",  
        ( b ? "==" : "!=" ) );  
  
    b = Arrays.equals( intArray, filledIntArray );  
    System.out.printf( "intArray %s filledIntArray\n",  
        ( b ? "==" : "!=" ) );  
} // end method printEquality...
```




CLASSES ABSTRACTAS

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

Bibliografia: Martins, F. Mário , "*Programação Orientada aos Objectos em Java2*", FCA,ISBN: 972-722-196-3

Eckel, Bruce, "*Thinking em Java*"
Prentice-Hall, Junho, 2000,
ISBN: 0-13-027363-5

CLASSES ABSTRACTAS

- ❑ São todas as classes nas quais pelo menos um, ou mesmo todos, os métodos de instância não se encontram implementados, mas tão só declarados sintacticamente (daí ser designados de abstractos ou simplesmente virtuais).
- ❑ As classes e métodos são declarados como sendo abstractos usando o qualificador ***abstract*** nas suas definições

CLASSES ABSTRACTAS

❑ Exemplo:

```
public abstract class Forma{  
    public abstract double area() ;  
    public abstract double perimetro() ;  
}
```

- ❑ De notar que, para que uma classe deva ser considerada abstracta, é suficiente que não tenha implementado apenas um dos seus métodos.
- ❑ Uma classe abstracta não pode criar instâncias.

CLASSES ABSTRACTAS

- ❑ Então para que servem estas classes?
 - ❑ As principais vantagens associadas a este tipo de classes é a possibilidade de desenvolver aplicações genéricas, facilmente adaptáveis a novos requisitos sem grandes implicações no código existente.

CLASSES ABSTRACTAS

- ❑ É de salientar que o mecanismo de herança se mantém em vigor mesmo que uma superclasse de um conjunto de subclasses seja uma classe abstracta.
- ❑ Assim, qualquer subclasse de uma classe abstracta herda automaticamente todos os métodos da classe abstracta, estejam estes implementados ou sejam abstractos

CLASSES ABSTRACTAS

- ❑ Uma classe abstracta, ao não implementar certos métodos, delega nas suas subclasses a implementação particular de tais métodos, facilitando o aparecimento de diferentes implementações dos mesmos métodos nas suas diferentes subclasses.
- ❑ Enquanto numa relação normal entre classes e subclasses a redefinição de métodos é opcional, na relação entre classes abstractas a definição de métodos é obrigatória.

CLASSES ABSTRACTAS

- ❑ O mecanismo de herança vai garantir que todas as subclasses da classe abstracta “falam” a mesma linguagem.
- ❑ Vantagens:
 - ❑ se todas as subclasses implementam os métodos abstractos, todas responderão a mensagens cujos nomes coincidem com tais identificadores de métodos. Deste modo, consegue-se uma importante garantia de normalização de vocabulário.

CLASSES ABSTRACTAS

❑ Vantagens:

- ❑ Quaisquer futuras subclasses devem igualmente ter que implementar os mesmos métodos, pelo que as suas instâncias responderão às mesma mensagens.

CLASSES ABSTRACTAS - EXEMPLOS

```
public abstract class Forma {  
    public abstract double area() ;  
    public abstract double perimetro() ;  
}
```

CLASSES ABSTRACTAS - EXEMPLOS

```
public class Rectangulo extends Forma {  
    public Rectangulo() {  
        comprimento = 0.0;  
        largura = 0.0;  
    }  
    public Rectangulo(double c, double l) {  
        comprimento = c;  
        largura = l;  
    }  
    private double comprimento, largura;  
    public double perimetro() {  
        return 2*(comprimento+largura);  
    }  
}
```

```
public double area() {  
    return comprimento * largura;  
}  
public double getLargura(){  
    return largura;  
}  
public double getComprimento(){  
    return comprimento;  
}  
}
```

CLASSES ABSTRACTAS - EXEMPLOS

```
import java.lang.Math;
public class Circulo extends Forma {
    public Circulo() {
        r = 1.0;
    }
    public Circulo(double r) {
        this.r = ((r <= 0.0) ? 1.0 : r);
    }

    private double r;
```

```
    public double perimetro() {
        return 2*Math.PI*r;
    }

    public double area() {
        return Math.PI*r*r;
    }

    public double getRaio(){
        return r;
    }
}
```

C... EXEMPLOS

```
public class Triangulo extends Forma {  
    public Triangulo() {  
        base = 0.0;  
        altura = 0.0;  
    }  
    public Triangulo(double b, double a) {  
        base = b;  
        altura = a;  
    }  
  
    private double base, altura;  
  
    public double perimetro() {  
        return base+altura+this.hipotenusa();  
    }  
}
```

```
    public double area() {  
        return base*altura/2;  
    }  
    public double getAltura(){  
        return altura;  
    }  
    public double getBase(){  
        return base;  
    }  
    public double hipotenusa(){  
        return Math.sqrt(  
            Math.pow(base,2.0)+  
            Math.pow(altura,2.0));  
    }  
}
```

CLASSES ABSTRACTAS - EXEMPLOS

```
public class Teste {  
    public static void main (String args[]) {  
        Triangulo forma1 = new Triangulo(2.0, 6.0);  
        Rectangulo forma2 = new Rectangulo(2.0, 13.0);  
        Circulo forma3 = new Circulo(3.0);  
  
        System.out.println("forma 1 = "+ forma1.getClass());  
        System.out.println("forma 2 = "+ forma2.getClass());  
        System.out.println("forma 3 = "+ forma3.getClass());  
        System.out.println("----- Triangulo-----");  
        System.out.println(forma1.getBase());  
        System.out.println(forma1.getAltura());  
        System.out.println(forma1.area());  
        System.out.println(forma1.perimetro());  
    }  
}
```

CLASSES ABSTRACTAS - EXEMPLOS

```
System.out.println("----- Rectangulo-----");  
System.out.println(forma2.getComprimento());  
System.out.println(forma2.getLargura());  
System.out.println(forma2.area());  
System.out.println(forma2.perimetro());  
System.out.println("----- Circulo-----");  
System.out.println(forma3.getRaio());  
System.out.println(forma3.area());  
System.out.println(forma3.perimetro());
```

```
}
```

```
}
```

CLASSES ABSTRACTAS - EXEMPLOS

```
public class Teste1 {  
    public static void main (String args[]) {  
        Forma forma1 = new Triangulo(2.0, 6.0);  
        Forma forma2 = new Rectangulo(2.0, 13.0);  
        Forma forma3 = new Circulo(3.0);  
  
        System.out.println("forma 1 = "+ forma1.getClass());  
        System.out.println("forma 2 = "+ forma2.getClass());  
        System.out.println("forma 3 = "+ forma3.getClass());  
        System.out.println("----- Triangulo-----");  
        System.out.println(forma1.area());  
        System.out.println(forma1.perimetro());  
    }  
}
```

CLASSES ABSTRACTAS - EXEMPLOS

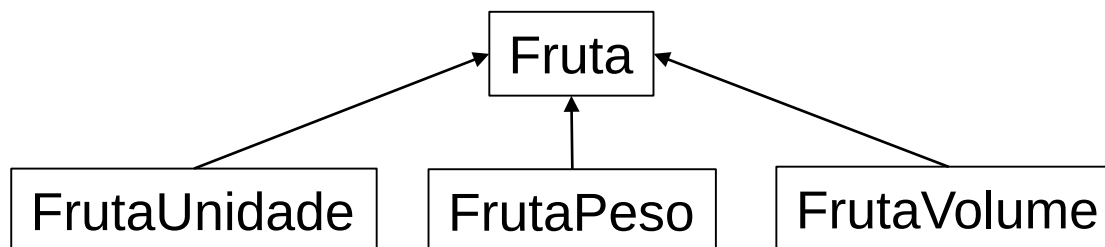
```
System.out.println("----- Rectangulo-----");  
System.out.println(forma2.area());  
System.out.println(forma2.perimetro());  
System.out.println("----- Circulo-----");  
System.out.println(forma3.area());  
System.out.println(forma3.perimetro());  
    }  
}
```


EXEMPLO DE IMPLEMENTAÇÃO

- ❑ Numa loja toda a fruta vendida tem um nome e um preço base. No entanto, a loja tem três tipos de frutos: à unidade, ao peso e ao volume.
- ❑ O preço total é obtido pela multiplicação do preço base pelo número de unidades, pelo peso, ou pelo volume.

EXEMPLO DE IMPLEMENTAÇÃO

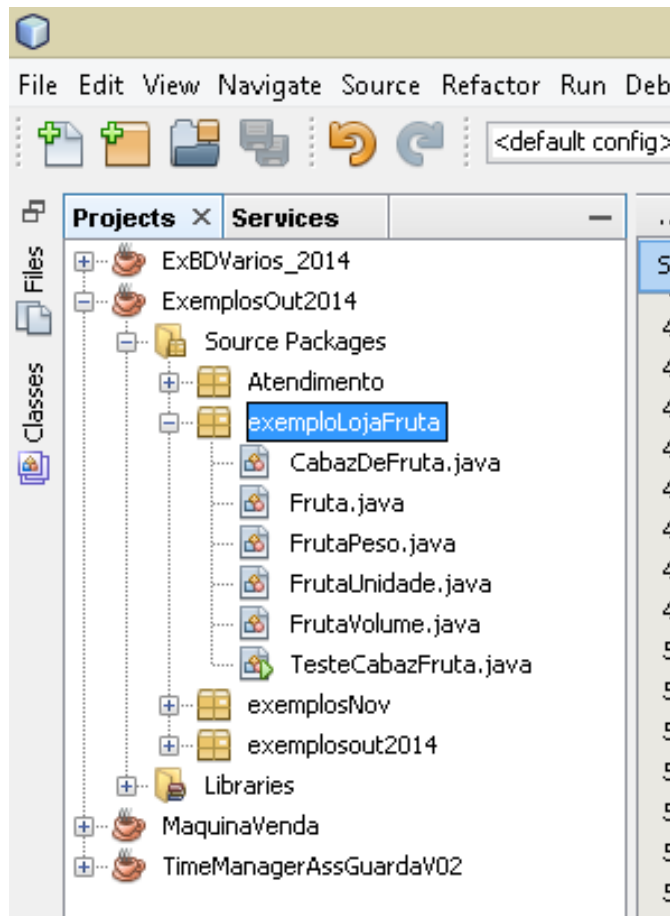
- ❑ Pretende-se criar uma hierarquia como na figura seguinte:



EXEMPLO

- ❑ É necessário criar também uma classe CabazDeFruta que representa as várias frutas e que implementa os métodos seguintes:
 - ❑ Método para inserir no cabaz uma compra
 - ❑ Método que calcule o valor total da compra
 - ❑ Método que determine o número de frutos comprados de um dado tipo
 - ❑ Método que determine o valor total gasto em cada tipo de fruta

EXEMPLO



EXEMPLO

ExemplosOut2014 - NetBeans IDE 8.

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

<default config>

Projects Services

Files

- ExBDVarios_2014
- ExemplosOut2014
 - Source Packages
 - Atendimento
 - exemploLojaFruta
 - CabazDeFruta.java
 - Fruta.java
 - FrutaPeso.java
 - FrutaUnidade.java
 - FrutaVolume.java
 - TesteCabazFruta.java
 - exemplosNov
 - exemplosout2014
 - Libraries
 - MaquinaVenda
 - TimeManagerAssGuardaV02

Classes

toString - Navigator

Members <empty>

Fruta

- Fruta()
- Fruta(String nome, double preco)

Source History

```
10  *
11  * @author JoseQuiterio
12  */
13  public abstract class Fruta {
14      public Fruta() {
15          nome = "";
16          preco = 0;
17      }
18      public Fruta(String nome, double preco) {
19          this.nome = nome;
20          this.preco = preco;
21      }
22      private String nome;
23      private double preco;
24
25      public abstract double aPagar();
26      public abstract String toString();
27
28
29      /**
30       * @return the nome
```

EXEMPLO

ExemplosOut2014 - NetBeans IDE 8.0

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

<default config>

Projects Services

Files

- ExBDVarios_2014
- ExemplosOut2014
 - Source Packages
 - Atendimento
 - exemploLojaFruta
 - CabazDeFruta.java
 - Fruta.java
 - FrutaPeso.java
 - FrutaUnidade.java
 - FrutaVolume.java
 - TesteCabazFruta.java
 - exemplosNov
 - exemplosout2014
 - Libraries
 - MaquinaVenda
 - TimeManagerAssGuardaV02

Classes

FrutaUnidade - Navigator

Members

- FrutaUnidade :: Fruta
 - FrutaUnidade(FrutaUnidade fu)
 - FrutaUnidade(String nome, double preco)
 - FrutaUnidade()
 - aPagar(): double
 - getQuantidade(): int
 - setQuantidade(int quantidade)
 - toString(): String
 - quantidade: int

```
13 public class FrutaUnidade extends Fruta {
14     public FrutaUnidade() {
15         super();
16         quantidade = 0;
17     }
18     public FrutaUnidade(String nome, double preco, int quantidade) {
19         super(nome, preco);
20         this.quantidade = quantidade;
21     }
22     public FrutaUnidade(FrutaUnidade fu) {
23         super(fu.getNome(), fu.getPreco());
24         quantidade = fu.getQuantidade();
25     }
26     private int quantidade;
27     @Override
28     public double aPagar() {
29         return quantidade * this.getPreco();
30     }
31
32     @Override
33     public String toString() {
34         StringBuilder output = new StringBuilder();
35         output.append("Nome = "+this.getNome() + "\n");
36         output.append("Preço Unidade = "+this.getPreco() + "\n");
37         output.append("Quantidade = "+ quantidade + "\n");
38         output.append("A Pagar = "+this.aPagar());
39         return output.toString();
40     }
41 }
```

EXEMPLO

ExemplosOut2014 - NetBeans IDE 8.0

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

...avz FrutaPeso.java x FrutaVolume.java x CabazDeFruta.java x TesteCabazFruta.java

Source History

```
15 public class CabazDeFruta {
16
17     public CabazDeFruta() {
18         cabaz = new ArrayList<Fruta>();
19     }
20
21     private ArrayList<Fruta> cabaz;
22
23     public void addFruta(Fruta f) {
24         cabaz.add(f);
25     }
26
27     public double valorTotal() {
28         double total = 0;
29         for (Fruta f : cabaz) {
30             total += f.aPagar();
31         }
32         return total;
33     }
34
35     public double valorTotalPorTipo(String tipoFruta) {
36         double total = 0;
37         for (Fruta f : cabaz) {
38             if (f.getClass().getSimpleName().equals(tipoFruta)) {
39                 total += f.aPagar();
40             }
41         }
42         return total;
43     }
44
45     public int numTotalPorTipo(String tipoFruta) {
```

Projects x Services

Files

- ExBDVarios_2014
- ExemplosOut2014
 - Source Packages
 - Atendimento
 - exemploLojaFruta
 - CabazDeFruta.java
 - Fruta.java
 - FrutaPeso.java
 - FrutaUnidade.java
 - FrutaVolume.java
 - TesteCabazFruta.java
 - exemplosNov
 - exemplosout2014
- Libraries
 - MaquinaVenda
 - TimeManagerAssGuardaV02

Classes

valorTotalPorTipo - Navigator x

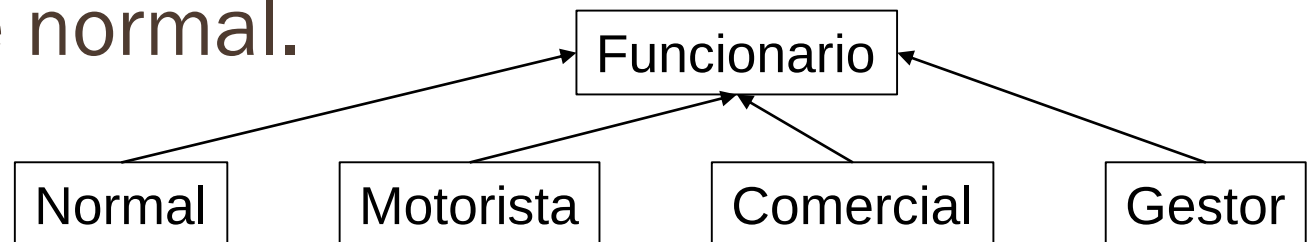
Members

<empty>

- CabazDeFruta
 - CabazDeFruta()
 - addFruta(Fruta f)
 - numTotalPorTipo(String tipoFruta) : int
 - valorTotal() : double
 - valorTotalPorTipo(String tipoFruta) : double
 - cabaz : ArrayList<Fruta>

EXEMPLO: EMPRESA

- ❑ Uma empresa possui uma ficha de cada funcionário contendo o nome, um código e, para cada mês, o número de dias de trabalho.
- ❑ A empresa tem um valor fixo por dia de trabalho. Porém, os funcionários são classificados como: gestor, motorista, comercial e normal.



EXEMPLO: EMPRESA

- ❑ Os gestores têm um prémio que é 15% do seu vencimento total.
- ❑ Os motoristas acumulam ao vencimento um bónus por quilómetros percorridos, fixado todos os anos.
- ❑ Os comerciais acumulam uma percentagem em função das vendas, valor fixado todos os anos.

EXEMPLO: EMPRESA

- ❑ Pretende-se criar uma classe Empresa que mantenha um registo atualizado dos seus funcionários.
- ❑ Operações a realizar:
 - ❑ Inserir uma ficha de funcionário
 - ❑ Verificar se existe algum funcionário com um código dado por parâmetro
 - ❑ Obter a ficha do funcionário dado o código, se existir
 - ❑ Devolver a lista dos funcionários existentes
 - ❑ Calcular o total dos salários a pagar
 - ❑ Determinar o número de gestores
 - ❑ Determinar o número de funcionários de uma categoria
 - ❑ Guardar uma lista dos funcionários num ficheiro de texto

EXEMPLO: EMPRESA

```
public abstract class Funcionario{  
    ...  
    private String codigo;  
    private String nome;  
    private int dias;  
    ...  
    public abstract double salario();  
}
```

EXEMPLO: EMPRESA

```
public class Motorista extends Funcionario  
public class Gestor extends Funcionario  
public class Comercial extends Funcionario  
public class Normal extends Funcionario
```

EXEMPLO: EMPRESA

```
public class Empresa {  
    private static double salarioDia = 60.0;  
    private static double valorKm = 0.025;  
    private static double comissão = 0.15;  
  
    ...  
    private ArrayList<Funcionario> tFuncionarios;  
  
    ...  
    public boolean existeFuncionario(String codigo);  
    public Funcionario daFicha(String codigo);  
    public void addFuncionario(Funcionario f);  
    public double totalSalarios();  
    public int numTotalDeTipo(String tipo);  
    public String toString();  
  
    ..  
}
```



INTERFACES JAVA

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

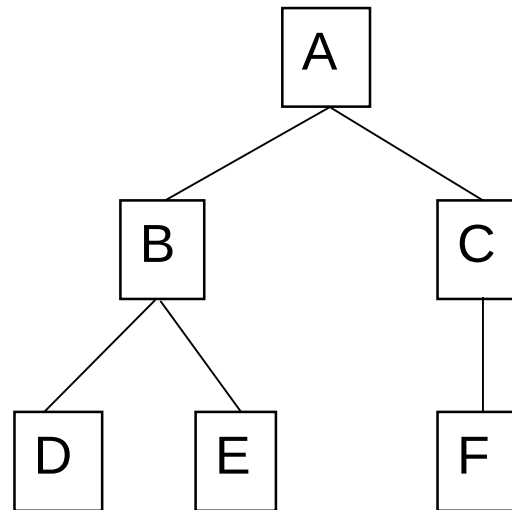
EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

Bibliografia: Martins, F. Mário , "*Programação Orientada aos Objectos em Java2*", FCA,ISBN: 972-722-196-3
Eckel, Bruce, "*Thinking em Java*"
Prentice-Hall, Junho, 2000,
ISBN: 0-13-027363-5

INTERFACES JAVA

- ❑ As Interfaces são um mecanismo extremamente útil de especificar e garantir que duas classes posicionadas em locais diferentes de uma hierarquia de classes, não tendo qualquer relação entre si, possuam comportamento comum.

- Suponhamos o exemplo seguinte:



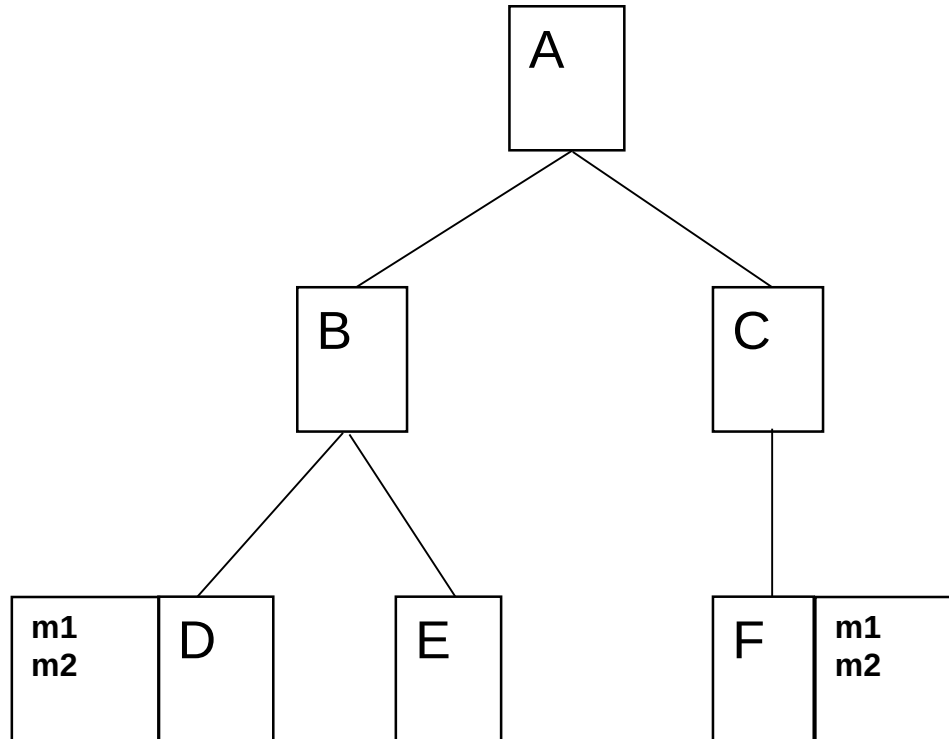
INTERFACES JAVA

- ❑ Da hierarquia exemplo apresentada as classes D e E possuem uma capacidade de responder a um conjunto de mensagens comum, que resulta da herança de B e A.
- ❑ Para além, de poderem juntar os seus métodos particulares.

INTERFACES JAVA

- ❑ As classes C e F, dado o seu posicionamento na hierarquia, já possuem linguagens distintas de D e E.
- ❑ Em comum tem apenas os métodos definidos na superclasse A.
- ❑ Suponhamos agora que pretendemos que as classes D e F, apenas estas, implementem os métodos m1 e m2.

INTERFACES JAVA



INTERFACES JAVA

- ❑ Perante este requisito, se a linguagem de POO usada apenas oferece ao programador o mecanismo de herança, então a solução passa por adicionar os métodos m1 e m2 à superclasse comum das duas classes, no exemplo seria na classe A.

INTERFACES JAVA

- ❑ Ter que alterar uma classe sempre que se pretender que algumas das suas mais distantes e não relacionadas subclasses implementem um dado comportamento, não é uma solução aceitável do ponto de vista de uma programação que se pretende estável, mas genérica e extensível.

INTERFACES JAVA

- ❑ A implementação de tal solução implica que todas as subclasses de A respondam de igual forma às mensagens.
- ❑ Isto contraria o objectivo inicial, de responderem unicamente a classes D e F.

INTERFACES JAVA

- ❑ A linguagem Java vai responder a este tipo de requisito utilizando um mecanismo adicional – as Interfaces.
- ❑ As Interfaces são especificações sintácticas de certos métodos que representam um comportamento particular, que qualquer classe, em qualquer ponto da hierarquia pode assumir com uma implementação sua.

INTERFACES JAVA

- ❑ O mecanismo de Interface permite ainda, a definição de mecanismos simples de comunicação e sincronização de estados entre classes, em particular entre classe de interface com o utilizador.
- ❑ Por exemplo, as opções a disponibilizar num menu num determinado contexto.

INTERFACES JAVA: DECLARAÇÃO

- ❑ Uma interface poderá ter apenas identificadores declarados como *static* e *final*, e um conjunto de métodos que são implicitamente abstractos, sendo opcional usar o qualificador *abstract*.
- ❑ As interfaces são semelhantes às classes abstractas, mas muito mais restritivas.

INTERFACES JAVA: DECLARAÇÃO

- Se uma classe B é subclasse de A, declaramos:

```
public class B extends A {
```

- Se para além da condição anterior a classe B implementa uma Interface I, deveremos declarar:

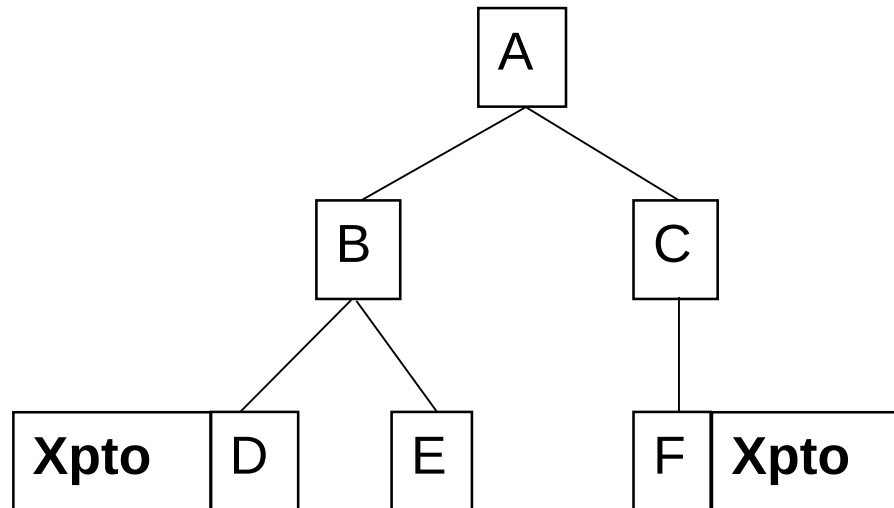
```
public class B extends A implements I {
```

INTERFACES JAVA: EXEMPLO

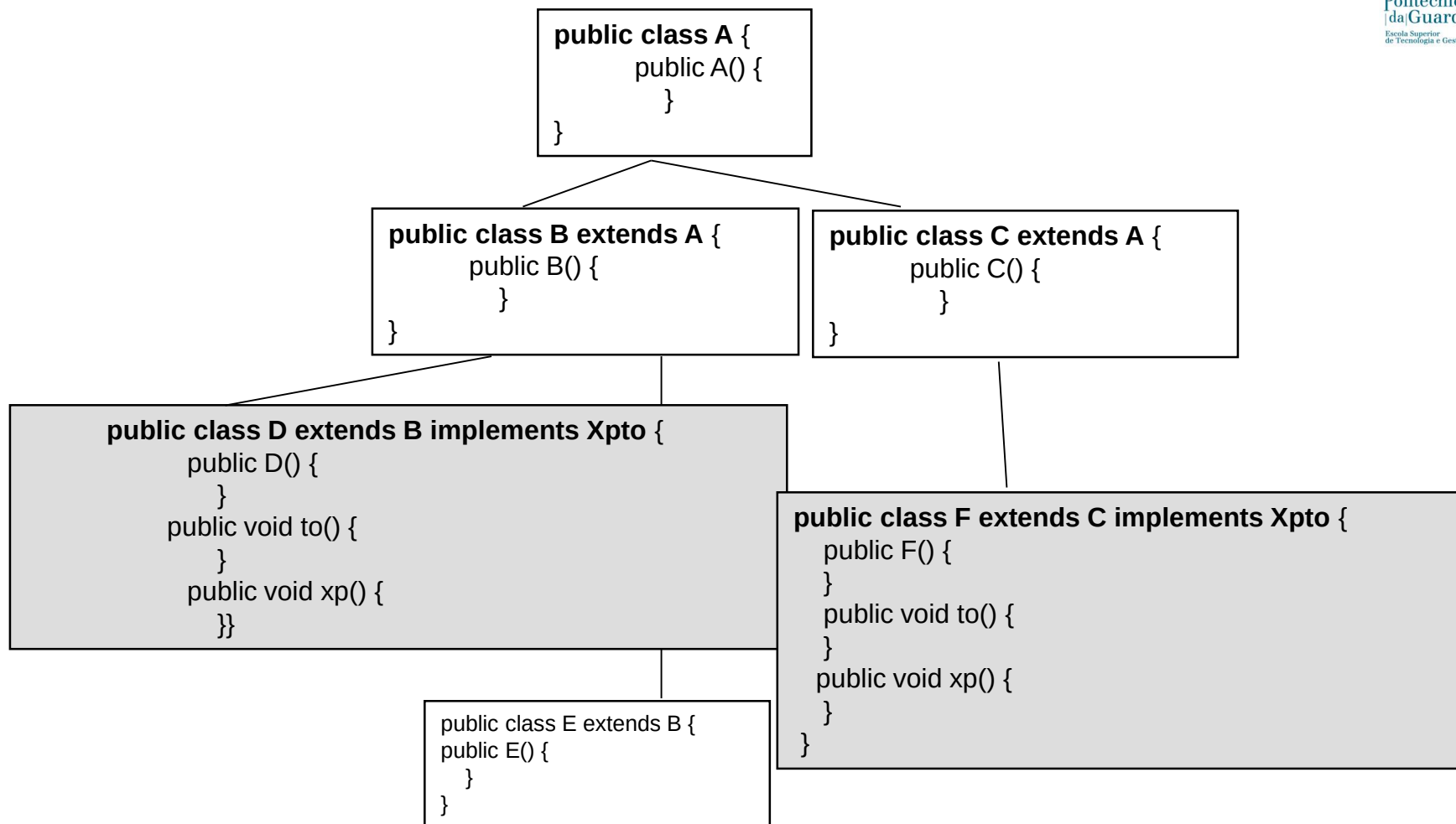
```
class interface Xpto{  
    public abstract void xp();  
    public void to();  
}
```

INTERFACES JAVA: EXEMPLO

□ Implementar o exemplo seguinte:



INTERFACES JAVA: EXEMPLO



UTILIZAÇÃO

- ❑ Existe um variado número de situações em engenharia de software, em que um conjunto de programadores totalmente distintos têm de estabelecer um “contrato” onde cada programador explicita como o seu software interage. Cada programador escreve o seu código, sem conhecimento algum de como os outros escrevem o seu código.
- ❑ As Interfaces são o “contrato”, onde está definido como o software interage.

INTERFACE EXEMPLO

```
public interface Bicycle {  
    void changeCadence(int newValue);  
    void changeGear(int newValue);  
    void speedUp(int increment);  
    void applyBrakes(int decrement);  
}
```

```
class ACMEBicycle implements Bicycle {  
    ....  
    // remainder of this class implemented as before  
    ...  
}
```

Experimentar a implementação...

INTERFACE COMO API

- ❑ Atualmente muitas empresas desenvolvem pequenos pacotes de software com funcionalidades muito específicas.
- ❑ Estas empresas vendem a outras que necessitam destas funcionalidades. Uma das formas de vender é fornecer a API, como devemos utilizar, mas manter a implementação secreta. Fazemos isto com a utilização de interfaces.

EXEMPLO DE UTILIZAÇÃO

- ❑ Vamos imaginar uma sociedade futurista onde o transporte de passageiros pela cidade é feito por veículos que são controlados por computadores sem qualquer intervenção humana.
- ❑ A indústria automóvel produz software em Java, que opera os automóveis – stop, start, accelerate, turn left.
- ❑ Outro grupo industrial, produz software em Java para um sistema de computadores receber dados GPS e transmitir via wireless as condições do tráfego rodoviário e utilizar essa informação no controlo automóvel.
- ❑ Os fabricantes de automóveis deve publicar uma interface padrão da indústria, que explica em detalhe que métodos podem ser chamados para fazer o movimento do veículo (qualquer veículo, de qualquer fabricante).
- ❑ A indústria que fornece a orientação pode escrever software que chama os métodos descritos na interface para comandar o carro. Nenhum grupo industrial precisa saber como o software de outro grupo é implementado. Cada grupo considera seu software secreto e reserva o direito de modificá-lo a qualquer momento, desde que continue a aderir à interface publicada.

INTERFACE COMO API

□ Exemplo

```
public interface OperateCar {
```

```
    // constant declarations, if any
```

```
    // method signatures
```

```
    // An enum with values RIGHT, LEFT
```

```
    int turn( Direction direction, double radius, double startSpeed, double endSpeed);
```

```
    int changeLanes( Direction direction, double startSpeed, double endSpeed);
```

```
    int signalTurn( Direction direction, boolean signalOn);
```

```
    int getRadarFront( double distanceToCar, double speedOfCar);
```

```
    int getRadarRear( double distanceToCar, double speedOfCar);
```

```
    // more method signatures
```

```
}
```

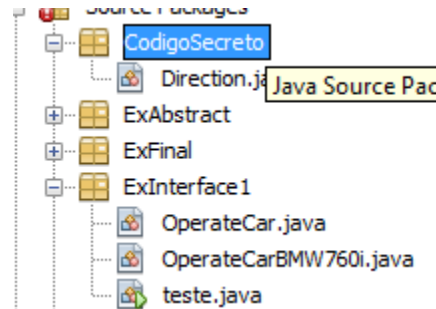
INTERFACE

```
package ExInterface1;
import CodigoSecreto.Direction;

public class OperateCarBMW760i implements OperateCar{
    public OperateCarBMW760i() {
    }
    public int turn(Direction direction, double radius, double startSpeed, double endSpeed) {
        System.out.println(""+direction.toString());
        return 1;
    }
    public int changeLanes(Direction direction, double startSpeed, double endSpeed) {}
    public int signalTurn(Direction direction, boolean signalOn) {}
    public int getRadarFront(double distanceToCar, double speedOfCar) {}
    public int getRadarRear(double distanceToCar, double speedOfCar) {}

}
```

INTERFACE



```
public static void main(String[] args) {  
    OperateCarBMW760i b = new OperateCarBMW760i();  
    Direction d = new Direction("Direita", 45, 5.6, 7.8);  
    System.out.println(""+b.turn(d, 45, 5.6, 7.8));  
}
```

INTERFACE

```
interface MinhasConstantes {  
    double PI = 3.141592;  
    double E = 2.7182818;  
}
```

```
public class Teste implements MinhasConstantes{
```

```
    public static void main(String[] args) {  
        System.out.println("Pi: "+ MinhasConstantes.PI);  
        System.out.println("E: "+E);  
    }
```

```
}
```

Se não declarar
implements MinhasConstantes

INTERFACE VS ABSTRACT CLASS

- ❑ Interface não é uma classe. Pode ser vista, como um tipo de operações, que qualquer classe pode utilizar desde que implemente esse interface.
- ❑ Uma classe que implemente uma interface tem de obedecer a duas condições:
 - ❑ Tem de especificar na definição da classe "implements *Interface_Name*";
 - ❑ Tem de implementar todos os métodos especificados na interface.

INTERFACE VS ABSTRACT CLASS

- ❑ As classes abstract estão inseridas numa hierarquia de classes, o que significa que existe uma forte relação entre elas.
- ❑ Por outro lado, a interface não existe necessariamente um relação entre classes.
- ❑ Interface é uma boa solução para a implementação de herança múltipla.
 - ❑ **Herança múltipla** é a capacidade de um objecto herdar características de vários objectos. Em Java não existe herança múltipla.

INTERFACE VS ABSTRACT CLASS

- ❑ Uma classe pode implementar vários interface.
- ❑ As classes abstract pode ter métodos com implementações.
- ❑ As interfaces apenas podem ter a definição, cabeçalho, dos métodos. Não podem ter corpo do método. Cada classe que implementa um interface é responsável pela sua implementação.

USAR INTERFACE OU ABSTRACT

- ❑ Usar abstract se planeamos uma hierarquia de classes.
- ❑ Usar abstract se planeamos utilizar métodos não públicos. Em Interface todos os métodos são públicos.
- ❑ Se prevemos que no futuro necessitamos de acrescentar novos métodos as class abstract são uma boa opção.
 - ❑ Se inserimos uma nova definição de método num interface, todas as classes que implementem este método têm de ser atualizadas.

USAR INTERFACE OU ABSTRACT

- ❑ As interfaces são uma boa opção quando prevemos que essa API não sofre alterações por bastante tempo.
- ❑ As interfaces são uma boa opção quando pretendemos implementar qualquer coisa semelhante à herança múltipla.

EXEMPLO RIDGESOFT, LLC

Using RoboJDE, create a new class named ContinuousRotationServo with the following code:

```
import com.ridgesoft.robotics.Motor;
import com.ridgesoft.robotics.Servo;

public class ContinuousRotationServo implements Motor {
    private Servo mServo;
    private boolean mReverse;
    private int mRange;
    private DirectionListener mDirectionListener;

    public ContinuousRotationServo(Servo servo,
                                   boolean reverse, int range) {
        mServo = servo;
        mReverse = reverse;
        mRange = range;
        mDirectionListener = null;
    }

    public ContinuousRotationServo(Servo servo,
                                   boolean reverse, int range,
                                   DirectionListener directionListener) {
        mServo = servo;
        mReverse = reverse;
        mRange = range;
        mDirectionListener = directionListener;
    }

    public void setDirectionListener(
        DirectionListener directionListener) {
        mDirectionListener = directionListener;
    }
}
```



IPG

Politécnico
da Guarda

Escola Superior
de Tecnologia e Gestão

EXCEPÇÕES

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

Bibliografia: Martins, F. Mário , "*Programação Orientada aos Objectos em Java2*", FCA,ISBN: 972-722-196-3
Eckel, Bruce, "*Thinking em Java*"
Prentice-Hall, Junho, 2000,
ISBN: 0-13-027363-5

EXCEPÇÕES

- ❑ O mecanismo de exceções é um dos pontos fortes da tão proclamada robustez do código dos programas em Java.
- ❑ O mecanismo de exceções em Java, deve ser capaz de permitir ao programador:
 - ❑ definir quais as exceções que devem ser detetadas num dado contexto do programa;
 - ❑ definir quais as ações que se pretendem ver executadas em tais situações;
 - ❑ definir as exceções a serem “lançadas” pelo próprio código do programa em certas situações;

O QUE É UMA EXCEPÇÃO?

- ❑ O termo **exception** é a abreviatura de "exceptional event"
- ❑ Uma exceção é um sinal gerado pela máquina virtual do Java, em tempo de execução do programa, e que é comunicado ao programa indicando a ocorrência de um ***erro recuperável***.
- ❑ São exemplos de exceções comuns:
 - ❑ tentativas de divisão por zero;
 - ❑ indexação para além dos limites de um array;
 - ❑ tentativas de leitura para além dos limites do ficheiro;

ERRO

- ❑ Um **erro** em Java corresponde a uma situação para a qual nenhuma recuperação é já possível, limitando-se o interpretador a enviar uma mensagem de erro e terminar a execução do programa.

EXCEPÇÕES

- ❑ Uma exceção é “lançada” (*thrown*) a partir do ponto do código onde a mesma ocorreu, e
- ❑ Diz-se “apanhada” (*catch*) no ponto do código para o qual o controlo de fluxo do programa foi transferido.

TIPO DE EXCEPÇÕES

❑ Implícito:

- ❑ automaticamente lançadas pela máquina virtual em função do erro detectado na execução.

❑ Explícito:

- ❑ quando é o próprio programador a codificar (detectar) o seu lançamento através de instrução própria.

TRATAMENTO DE EXCEPÇÕES

□ *try e catch*

- A palavra ***try*** precede em geral um bloco de instruções considerado pelo programador como contendo uma ou várias instruções que podem gerar erros de execução.
- Alguns dos possíveis erros de execução são muito difíceis de determinar, pelo que se sugere a inclusão deste tipo de operações.

TRATAMENTO DE EXCEPÇÕES

- ❑ O que é possível fazer ao nível do código para que este não termine de imediato a sua execução, e possa realizar o tratamento do erro e recuperar, se tal for possível, o fluxo normal de execução?
- ❑ A resposta está na cláusula ***catch***.
 - ❑ Esta palavra está indexada pelo identificador de um exceção particular, que identifica qual a exceção que pode ocorrer no bloco de instruções da cláusula *try*.

TRATAMENTO DE EXCEPÇÕES

- ❑ E também por um bloco de instruções que tem por objetivo a codificação das diversas instruções que irão corresponder às várias operações de recuperação/tratamento do estado do programa, visando a continuação correta do programa após os erros que foram, por tais mecanismos, detetados e corrigidos, caso seja possível.

TRATAMENTO DE EXCEPÇÕES

- ❑ Estrutura genérica para a captura e tratamento das exceções:

try

```
{ // O Código do programa tal como seria escrito mesmo que garantidamente não pudesse  
  // gerar qualquer erro, é colocado neste bloco. Porém, ao colocá-lo num bloco try passaremos  
  // a ter a possibilidade de detectar a ocorrência de alguns possíveis erros.  
}
```

catch(Identificador_da_excepção var_exc1)

```
{ // Aqui é escrito o código de tratamento da exceção identificada na cláusula catch, sendo  
  // var_exc1 a instância da exceção que foi gerada.  
}
```

catch(Identificador_da_excepção var_exc2)

```
{ // Poderemos ter várias cláusulas catch para o mesmo bloco try, cada uma correspondendo  
  // a uma classe de exceção diferente. }
```

finally

```
{ // O código aqui colocado será sempre executado, surja ou não uma exceção em try  
  // Este código pode ser muito útil para, por exemplo, fechar ficheiro e libertar recursos.  
  // Se a forma de saída do bloco try, for return, continue ou break, o bloco finally é de  
  // imediato executado. }
```

TRATAMENTO DE EXCEPÇÕES: EXEMPLO

```
import java.lang.*;
public class Teste1 {
    private static String leStr() {
        int ch;
        String s = "";
        boolean fim = false;
        while(!fim) {
            try {
                ch = System.in.read();
                if ((ch<0) || ((char) ch =='\n'))
                    fim = true;
                else
                    s += (char) ch; }
            catch(java.io.IOException e) {
                fim = true; }
        }
        return s;
    } // Fim leStr()
}
```

TRATAMENTO DE EXCEPÇÕES: EXEMPLO

```
public static void main (String args[]) {
```

```
    boolean fim = false;    int index;
```

```
    String palavra = "";
```

```
    String [] tabPalavras = new String[10];
```

```
    System.out.print("Int. uma palavra : ");
```

```
    System.out.flush();
```

```
    palavra = leStr();
```

```
    while(!palavra.equals(" "))
```

```
    {
```

```
        System.out.print("Introduza um indice : ");
```

```
        System.out.flush();
```

```
        index = Integer.valueOf(leStr().trim()).intValue();
```

```
        tabPalavras[index] = palavra;
```

```
        System.out.print("Int. uma palavra : ");
```

```
        System.out.flush();
```

```
        palavra = leStr();
```

```
    }
```

```
    } // Fim main
```

```
    } // Fim class Teste1
```

```
try{
```

```
    index = Integer.valueOf(leStr().trim()).intValue();
```

```
    tabPalavras[index] = palavra;
```

```
}catch (ArrayIndexOutOfBoundsException e){
```

```
    System.out.println("Índice fora dos limites !!!");}
```

```
catch (NumberFormatException e){
```

```
    System.out.println("Inteiro inválido !!!");}
```

LANÇAMENTO DE EXCEPÇÕES: *THROW E THROWS*

- ❑ Por vezes é necessário que o programador lance explicitamente uma exceção no meio da execução do código normal de um método.
- ❑ Para tal, o programador deve usar a cláusula ***throw***, em conjunto com a criação de uma instância particular da exceção pretendida, usando *new*.

LANÇAMENTO DE EXCEPÇÕES: *THROW E THROWS*

- ❑ Porém, a linguagem Java requer que qualquer método que possa provocar a ocorrência de uma exceção normal possua uma declaração ***throw***,
- ❑ Ou faça localmente o tratamento de tal exceção numa cláusula ***catch***,
- ❑ Ou declare explicitamente que pode lançar tal exceção, embora não a trate localmente, declarando no cabeçalho a cláusula ***throws***.

LANÇAMENTO DE EXCEPÇÕES: *THROW E THROWS: EXEMPLO*

```
public void pop() throws EmptyStackException{  
    if (this.empty())  
        throw new EmptyStackException();  
    else  
        numElem -= 1;  
}
```



I/O EM JAVA

Bibliografia:

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

[1] Martins, F. Mário , "*Programação Orientada aos Objectos em Java2*", FCA, ISBN: 972-722-196-3

[2] Mendes, António José; Marcelino, Maria José; "*Fundamentos de Programação em Java2*", FCA, ISBN: 972-722-267-6

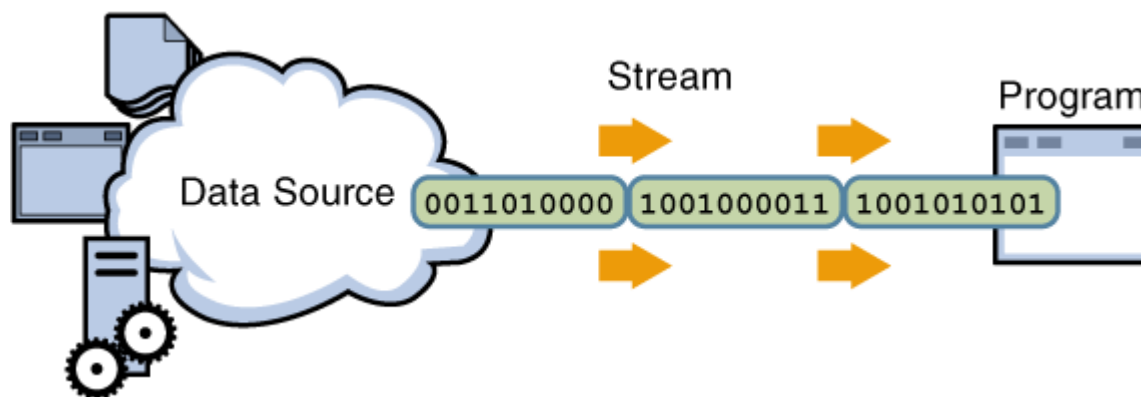
[3] Eckel, Bruce, "*Thinking em Java*" Prentice-Hall, Junho, 2000, ISBN: 0-13-027363-5

I/O EM JAVA

- ❑ Esta secção, tem por objectivo introduzir as técnicas e mecanismos de Java para a leitura e a escrita de dados de e para um qualquer dispositivo.
- ❑ Em Java, todo o que se relaciona com as mais diferentes formas de realizar a leitura e a escrita está associado ao conceito de ***stream***.

STREAM

- Uma *stream* é uma abstracção que representa uma fonte genérica de entrada de dados ou um destino genérico para escrita de dados, de acesso sequencial e independente de dispositivos físicos ou formatos de leitura e escrita.



Adaptada de <http://download.oracle.com/javase/tutorial/essential/io/streams.html>

STREAM

- ❑ Como todas as abstracções terá de ser refinada e concretizada e ser associada a uma entidade física de suporte de dados, seja um ficheiro em disco ou em CDRom, um WebSite, um array de bytes, uma string, etc ...

STREAM

- ❑ Existem duas grandes classes de streams:
 - ❑ streams de caracteres, ou seja, streams de texto;
 - ❑ streams de bytes, ou seja, streams binárias.

- ❑ Normalmente, um objecto a partir do qual seja possível ler sequências de bytes ou caracteres será um ***input stream***.
- ❑ Igualmente, um qualquer objecto no qual possa ser escritas sequências de bytes ou de caracteres será uma ***output stream***.

STREAM

- ❑ Todas as classes que irão implementar formas particulares de I/O em streams, serão subclasses de quatro classes abstractas do *package* **java.io**.
- ❑ Para as streams de bytes, as classes:
 - ❑ InputStream
 - ❑ OutputStream
- ❑ Para as streams de caracteres, as classes:
 - ❑ Reader
 - ❑ Writer

I/O BÁSICO

- ❑ A classe *java.lang.System* oferece serviços básicos de I/O através das variáveis de classe: *in*, *out* e *err*.
- ❑ Estas variáveis são, de facto, instâncias de streams que estão associadas, respectivamente, a operações de leitura a partir do teclado e de escrita no monitor.
- ❑ Daí termos já usado instruções do tipo:
 - ❑ `System.out.print("Ola");`
 - ❑ `System.in.read();`

PACKAGE JAVA.IO

❑ Classes não *stream*:

- ❑ Classe `File`;
- ❑ Classe `RandomAccessFile`;

❑ Classe *File*

- ❑ Esta classe representa os usuais ficheiros e directorias de um sistema de ficheiros.

CLASSE *FILE*

- ❑ Esta classe define métodos para a realização das usuais operações sobre ficheiros e directorias, tais como:
 - ❑ a listagem de directorias;
 - ❑ apagar ficheiros;
 - ❑ determinar atributos de um ficheiro;
 - ❑ alterar o seu nome;
 - ❑ etc...

CLASSE *FILE* : EXEMPLO

```
import java.io.File;
public class ClassFile {

    public static void main (String args[]) {

        File f1 = new File("teste.txt");
        System.out.println(f1.getName());
        System.out.println(f1.getPath());
        System.out.println(f1.isFile());
        System.out.println(f1.isDirectory());
        System.out.println(f1.length());

    }

}
```

CLASSE RANDOMACCESSFILE

- ❑ Esta classe implementa métodos de leitura e escrita em ficheiros de uma forma aleatória.
- ❑ Os tipos de dados a serem lidos ou escritos podem ser bytes, texto e tipos primitivos de Java, mas não objectos.
- ❑ A indicação da leitura e escrita é feita a muito baixo nível, é indicada a deslocação, em termos de bytes, a partir do início do ficheiro.

CLASSE RANDOMACCESSFILE : EXEMPLO

```
import java.io.RandomAccessFile;
public class RandomFile {
    public static void main (String args[]) {
        int datarray[] = {12,31,56,23,27,1,43,65,4,99};
        try{
            RandomAccessFile rf = new RandomAccessFile("temp.dat","rw");
            // Escrever no ficheiro
            for (int i=0;i<datarray.length;i++)
                rf.writeInt(datarray[i]);
            // Ler do ficheiro por ordem inversa
            for (int i=datarray.length-1;i>=0;i--) {
                rf.seek(i*4); // Int * 4 bytes
                System.out.println(rf.readInt());
            }
            rf.close();
        }
        catch(java.io.IOException e)
        { System.out.println("Erro no acesso ao ficheiro!!!");}
    }
}
```

CLASSE RANDOMACCESSFILE

- ❑ Devido ao baixo nível dos seus serviços, a classe *RandomAccessFile* é em geral pouco utilizada.
- ❑ A vantagem inerente à utilização de este tipo de ficheiros acontece quando os registos são de comprimentos fixo, onde se junta também o facto de permitirem o acesso simultâneo a leitura e escrita.

AS STREAMS DE JAVA

- ❑ Vamos começar por analisar as classes:
 - ❑ Writer
 - ❑ Reader
- ❑ Estas classes definem o comportamento de um tipo especial de *streams* - ***streams de caracteres.***

EXEMPLO

- ❑ Para auxiliar no estudo que vamos realizar sobre as streams em Java, vamos criar um classe designada ***FichaAluno***, cujas instâncias irão representar a ficha individual de um aluno, contendo:
 - ❑ nome, idade, curso, ano e lista de disciplinas.

EXEMPLO

```
import java.util.*;
import java.io.*;
public class FichaAluno {
    public FichaAluno (String nome, int idade,
                       int ano, String curso,
                       Vector discp) {
        this.nome = nome; this.idade = idade;
        this.ano = ano; this.curso = curso;
        this.inscricao = discp;
    }

    // variaveis de instancia
    private String nome;
    private int idade;
    private String curso;
    private int ano;
    private Vector inscricao;
```

EXEMPLO

```
// metodos de instancia
```

```
public String daNome() { return this.nome; }  
public int daIdade() { return this.idade; }  
public String daCurso() { return this.curso; }  
public int daAno() { return this.idade; }  
public Vector daInsc() {  
    return (Vector) this.inscricao.clone();  
}  
  
public Object clone() {  
    return new FichaAluno(this.nome, this.idade,  
                           this.ano, this.curso,  
                           this.daInsc());  
}
```

EXEMPLO

```
public String toString() {  
    StringBuffer ficha = new StringBuffer("-----");  
    ficha.append("\n");  
    ficha.append("Nome: "); ficha.append('\t');  
    ficha.append(this.nome); ficha.append("\n");  
    ficha.append("Idade: "); ficha.append('\t');  
    ficha.append(this.idade); ficha.append("\n");  
    ficha.append("Curso: "); ficha.append('\t');  
    ficha.append(this.curso); ficha.append("\n");  
    ficha.append("Ano: "); ficha.append('\t');  
    ficha.append(this.ano); ficha.append("\n");  
    int numDiscp = this.inscricao.size();  
    ficha.append("Disciplinas: "); ficha.append('\t');  
    ficha.append(numDiscp); ficha.append("\n");  
    for(int i = 0; i < numDiscp; i++) {  
        ficha.append(inscricao.elementAt(i));  
        ficha.append("\n");  
    }  
    return ficha.toString();  
}
```

EXEMPLO

```
public String asString() { // para comparacao com toString()
    String ficha = "-----" + "\n";
    ficha = ficha + "Nome: " + "\t" + this.nome + "\n";
    ficha = ficha + "Idade: " + "\t" + this.idade + "\n";
    ficha = ficha + "Curso: " + "\t" + this.curso + "\n";
    ficha = ficha + "Ano: " + "\t" + this.ano + "\n";
    int numDiscp = this.inscricao.size();
    ficha = ficha + "Disciplinas: " + "\t" + numDiscp + "\n";
    for(int i = 0; i < numDiscp; i++) {
        ficha = ficha + inscricao.elementAt(i) + "\n";
    }
    return ficha;
}
} // Fim da Classe FichaAluno
```

EXEMPLO

- Tendo por base esta classe *FichaAluno*, vamos estudar as várias possibilidades existentes em Java para se realizar I/O, analisando quer a funcionalidade quer a eficiência.

CLASSE WRITER

- ❑ Esta classe é a superclasse de todas as streams de caracteres.
- ❑ Especifica um conjunto básico de métodos para a escrita de caracteres que qualquer implementação de uma stream de caracteres deverá oferecer.
- ❑ *...Ver Class Writer...*

CLASSE WRITER

- ❑ As diversas subclasses de *Writer*, *Reader*, *InputStream* e *OutputStream*, possuem designações iniciadas por prefixos tais como:
 - ❑ *Print*, *Buffered*, *File*, *Piped*, *CharArray*, *String* e outros;
- ❑ que visam dar uma ideia do tipo de implementação que foi realizada.
 - ❑ Exemplo: *BufferedWriter*, *FileWriter*, ...

CLASSE WRITER

- ❑ Existe uma certa distinção entre a stream lógica que nos interessa considerar e a stream física que, ao nível do sistema operativo, está de facto a ser por nós lida ou escrita, seja através de caracteres ou de bytes.
- ❑ Por vezes, temos que criar uma instância da stream lógica que pretendemos usar, mas associando-a fisicamente à stream Java que implementa a manipulação de uma fonte, ou de um destino físico, que é uma string, ou como neste caso uma *StringReader* ou *StringWriter*.

CLASSE WRITER

- ❑ Por este facto, a maioria das vezes utiliza-se associações entre streams lógicas e físicas, como é o caso de:
*BufferedWriter bufout = new BufferedWriter (
new FileWriter("nomeficheiro.dat"));*
- ❑ Neste exemplo, utiliza-se um mecanismo de *buffering* para a escrita de caracteres num dado destino.
- ❑ O Programador envia mensagens à instância *BufferedWriter*, que serve de interface automática para a manipulação da *FileWriter* que lhe foi associada.

CLASSE WRITER: EXEMPLO

```
import java.io.*;
import java.util.*;

public class TstFichaAluno1 {
    public static void main(String args[]) {
        final int MAXFICHAS = 300000;
        // Disciplinas
        Vector disc = new Vector();
        disc.insertElementAt(new String("PI"), 0);
        disc.insertElementAt(new String("PII"), 1);
        disc.insertElementAt(new String("PIII"), 2);
        disc.insertElementAt(new String("AED"), 3);
        disc.insertElementAt(new String("Fisica"), 4);

        // Cria 300000 instancias diferentes mas de igual valor !!!
        FichaAluno a1 = new FichaAluno("José Fulano", 21, 2, "EI", disc);
        FichaAluno[] turma = new FichaAluno[MAXFICHAS];
        for(int i=0; i < MAXFICHAS-1; i++) {
            turma[i] = (FichaAluno) a1.clone();
        }
    }
}
```

CLASSE WRITER: EXEMPLO

```
String ficha;  
// Grava as MAXFICHAS fichas numa FileWriter  
// via uma BufferedWriter  
GregorianCalendar c = new GregorianCalendar();  
Date d = c.getTime(); System.out.println(d);  
try {  
    BufferedWriter bout =  
        new BufferedWriter(new FileWriter("fich1.dat"));  
    for(int i=0; i < MAXFICHAS-1; i++) {  
        ficha = turma[i].asString();  
        bout.write(ficha);  
    }  
    bout.flush(); bout.close();  
}  
catch(IOException e) { System.out.println(e.getMessage()); }  
c = new GregorianCalendar();  
d = c.getTime(); System.out.println(d);  
}  
}
```

VERIFICAR E ANALISAR OS DIVERSOS EXEMPLOS POSSÍVEIS:

```
FileWriter fout = new FileWriter("fich2.dat");
```

```
PrintWriter fout =new PrintWriter(new FileWriter("fich3.dat"));
```

```
PrintWriter fout = new PrintWriter(new FileOutputStream("fich4.dat"));
```

```
DataOutputStream dout = new DataOutputStream(new FileOutputStream("fich5.dat"));
```

```
PrintWriter fout = new PrintWriter(new FileOutputStream("alunos6.dat"));
```

```
ObjectOutputStream oout = new ObjectOutputStream(new FileOutputStream("fich7.dat"));
```

CLASSE READER

- ❑ Trata-se de uma classe abstracta que define um conjunto de métodos para a leitura de streams de caracteres;
- ❑ Faz a leitura de um caracter, *int read()*, ou de um array de caracteres, *int read(char []cbuf)*, e que devolve o valor -1 ao ser atingido o fim da stream.

CLASSE READER

- Esta classe é simétrica da classe *InputStream* para stream de bytes, e as suas subclasses são simétricas das subclasses de *Writer*.

CLASSE READER: EXEMPLO

- ❑ Analisando o nosso exemplo, a leitura de dados deve compreender não só a leitura dos dados, mas também a reconstituição dos dados. Uma vez que foram adicionados dados para formatação que agora devem ser desprezados.

CLASSE READER: EXEMPLO

- ❑ Para o primeiro exemplo, *fich1.dat*, no qual foi usado uma *BufferedWriter* sobre um *FileWriter*, e tirando partido da apregoada simetria entre as classes, vamos usar uma *BufferedReader* sobre um *FileReader*.

CLASSE READER: EXEMPLO

- ❑ A classe *FileReader* permite que sejam lidas sucessivas linhas do ficheiro texto utilizando o método ***readLine()***.
- ❑ Porém, temos para além de ler os dados do ficheiro extrair a informação útil.

CLASSE READER: EXEMPLO

- ❑ Em Java, existe uma classe designada por ***StringTokenizer*** que permite a extracção de *tokens* (palavras) a partir de uma *string*, desde que estas estejam separadas por espaços, *tabs* ou *newlines*.
- ❑ Analisar a implementação efectuada no exemplo: ***TstLeFichaAluno1***.

EXEMPLO: TSTLEFICHAALUNO1

```
import java.io.*;
import java.util.*;

public class TstLeFichaAluno1 {

    public static void main(String args[]) {
        final int MAXFICHAS = 300000;

        FichaAluno[] turma = new FichaAluno[MAXFICHAS];
        FichaAluno ficha;

        // Le as MAXFICHAS de uma FileReader via uma BufferedReader
        try {
            String linha;
            StringTokenizer st;
            String txt, nome, curso, dcp;
            int ano, idade, discp;
            GregorianCalendar c = new GregorianCalendar();
            Date d = c.getTime();
            System.out.println(d);
            BufferedReader bin = new BufferedReader(new FileReader("fich1.dat"));
        }
    }
}
```

EXEMPLO: TSTLEFICHAALUNO1

```
for(int i=0; i < MAXFICHAS-1; i++) {  
    // le uma ficha de aluno  
    linha = bin.readLine(); //despreza separador  
    // NOME: .....  
    st = new StringTokenizer(bin.readLine(), "\\t");  
    txt = st.nextToken();  
    nome = st.nextToken();  
    // Idade: .....  
    st = new StringTokenizer(bin.readLine(), "\\t");  
    txt = st.nextToken();  
    idade = Integer.valueOf(st.nextToken()).intValue();  
    // Curso: .....  
    st = new StringTokenizer(bin.readLine(), "\\t");  
    txt = st.nextToken();  
    curso = st.nextToken();  
    // Ano: .....  
    st = new StringTokenizer(bin.readLine(), "\\t");  
    txt = st.nextToken();  
    ano = Integer.valueOf(st.nextToken()).intValue();
```

```
// Disciplinas: .....  
st = new StringTokenizer(bin.readLine(), "\\t");  
txt = st.nextToken();  
discp = Integer.valueOf(st.nextToken()).intValue();  
// ciclo para as disciplinas  
Vector disciplinas = new Vector();  
for(int ds=0; ds < discp; ds++) {  
    st = new StringTokenizer(bin.readLine());  
    txt = st.nextToken();  
    disciplinas.insertElementAt(txt, ds);  
}  
ficha = new FichaAluno(nome, idade, ano, curso, disciplinas);  
System.out.println(i);  
turma[i] = ficha;  
}  
bin.close();  
c = new GregorianCalendar();  
d = c.getTime();  
System.out.println(d);  
// apresenta as primeiras 4 fichas das MAXFICHAS lidas  
for(int x=0; x < 4; x++) {  
    System.out.println(turma[x].toString());  
}  
}  
catch(IOException e) { System.out.println(e.getMessage()); }  
}
```



FICHEIROS

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

Bibliografia:

*Mendes, António José; Marcelino, Maria José;
“Fundamentos de Programação em Java2”,
FCA, ISBN: 972-722-267-6*

FICHEIROS DE TEXTO

- ❑ No sentido de concretizar melhor os conceitos associados à utilização de ficheiros, vamos criar um classe que permita manipular ficheiros de texto
- ❑ A utilização de ficheiros normalmente envolve 4 operações básicas:
 - ❑ Abertura do ficheiro;
 - ❑ Leitura e escrita;
 - ❑ Fecho do ficheiro

FICHEIROS DE TEXTO

- ❑ Para manipular o ficheiro de texto devidamente, vamos utilizar as classes:
 - ❑ BufferedReader
 - ❑ BufferedWriter

```
import java.io.*;  
public class FicheirodeTexto {  
    private BufferedReader fR;  
    private BufferedWriter fW;  
}
```


FICHEIROS DE TEXTO

□ Abertura do ficheiro para leitura:

```
public void abreLeitura(String nomeFicheiro)  
throws IOException {  
    fR = new BufferedReader(new FileReader(nomeFicheiro));  
}
```

□ Abertura do ficheiro para escrita:

```
public void abreEscrita(String nomeFicheiro)  
throws IOException {  
    fW = new BufferedWriter(new FileWriter(nomeFicheiro));  
}
```

FICHEIROS DE TEXTO

❑ Leitura de uma linha do ficheiro:

```
public String lerLinha() throws IOException {  
    return fR.readLine();  
}
```

❑ Escrita de uma linha no ficheiro:

```
public void escreverLinha(String linha) throws IOException {  
    fW.write(linha,0,linha.length());  
    fW.newLine();  
}
```

FICHEIROS DE TEXTO

- ❑ 0 métodos para fechar os ficheiros:

```
public void fechaLeitura() throws IOException {  
    fR.close();  
}
```

```
public void fechaEscrita() throws IOException {  
    fW.close();  
}
```

FICHEIROS DE OBJECTOS

- ❑ A leitura e armazenamento de objectos de e para ficheiros são assegurados pelas classes:
 - ❑ `ObjectInputStream`
 - ❑ `ObjectOutputStream`
- ❑ Estas classes através dos métodos:
 - ❑ `writeObject()` e `readObject()`
- ❑ Organizam e reorganizam os dados dos objectos de modo a que possam ser escrito e lidos de um ficheiro.

FICHEIROS DE OBJECTOS

- ❑ No caso do programador pretender armazenar em ficheiro objectos de classes definidas por si, terá que indicar expressamente que autoriza a sua reorganização para armazenamento em ficheiro.
- ❑ Para isso deve acrescentar ao cabeçalho dessas classes as palavras:

implements Serializable

FICHEIROS DE OBJECTOS

- ❑ Classe de demonstração da utilização de ficheiros de objectos:

```
import java.io.*;  
public class FicheiroDeObjectos{  
    private ObjectInputStream iS;  
    private ObjectOutputStream oS;  
}
```

FICHEIROS DE OBJECTOS

- ❑ Método para abertura do ficheiro para leitura:

```
public boolean abreLeitura(String nomeDoFicheiro) {  
    try{  
        iS = new ObjectInputStream(new FileInputStream(nomeDoFicheiro));  
        return true;  
    } catch( IOException e ){ return false; }  
}
```

- ❑ Método para abertura do ficheiro para escrita:

```
public void abreEscrita(String nomeDoFicheiro) throws IOException{  
    oS = new ObjectOutputStream(new FileOutputStream(nomeDoFicheiro));  
}
```

FICHEIROS DE OBJECTOS

- ❑ Método para leitura de um objecto do ficheiro:

```
public Object leObjecto( ) throws IOException{  
    return iS.readObject();  
}
```

- ❑ Método para escrita de um objecto no ficheiro:

```
public void escreveObjecto(Object o) throws IOException{  
    oS.writeObject(o);  
}
```


FICHEIROS DE OBJECTOS

❑ Métodos para fechar o ficheiro:

```
public void fechaLeitura( ) throws IOException{  
    iS.close();  
}
```

```
public void fechaEscrita() throws IOException{  
    oS.close();  
}
```



IPG

Politécnico
da Guarda

Escola Superior
de Tecnologia e Gestão

JDBC

JOSÉ QUITÉRIO

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

EMAIL: [Mailto:JFIG@ipg.pt](mailto:JFIG@ipg.pt)

Bibliografia:

The Java Tutorial, java.sun.com
Programação em Java 2
Pedro Coelho - FCA - Editora de Informática
1001 Java Programmers Tips
Apontamentos da disciplina “Programação de
Aplicações Distribuídas”, por José Luís Oliveira,
DET/UA

- ❑ JDBC - Java DataBase Connectivity
- ❑ JDBC é uma interface SQL para acesso a bases de dados.
- ❑ Para se poder efectuar a ligação entre a aplicação JAVA e uma determinada base de dados, é necessário existir um *driver* específico que faça a ligação entre o JDBC e a base de dados em causa.

- ❑ O grande sucesso do JAVA, fez com que os fabricantes das principais bases de dados disponibilizem *drivers* JDBC para as suas bases de dados.
- ❑ A *Sun* criou o JDBC API, uma especificação com um conjunto de classes, interfaces e métodos, com o objectivo de ter uma interface única e independente da base de dados para as suas aplicações JAVA.

- ❑ No entanto, a existência de muitas aplicações de tipo Microsoft, que utiliza os drivers **ODBC** para a ligação a bases de dados Microsoft, existe também no JDK um produto chamado **JDBC-ODBC bridge**, que transforma as chamadas JDBC em ODBC.

JDBC - LIGAÇÃO

- ❑ Para estabelecer uma ligação são necessários alguns procedimentos:
 - ❑ 1. Carregar o driver adequado
`Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");`
 - ❑ 2. Criar a ligação à base de dados
`String url = "jdbc:odbc:namedatabase";`
`Connection con = DriverManager.getConnection(url, "login";
"passwd");`
 - ❑ 3. Criar um statement
`Statement stmt = con.createStatement();`
 - ❑ 4. Executar e ler resultados

JDBC - COMANDOS SOBRE A BD

- ❑ Construção do comando

String query = “SELECT * FROM Alunos”;

- ❑ É enviada a mensagem *executeQuery* ao objecto do tipo *Statement*

ResultSet rs = stmt.executeQuery(query);

- ❑ Tratamento da informação em *rs*

JDBC - TRATAMENTO DA INFORMAÇÃO

❑ Para tratar separadamente os campos de **rs**:

- ❑ `rs.next()` // muda de linha
- ❑ `rs.getString("campo")` // fornece o "campo" na linha
- ❑ `rs.getString(coluna)`

❑ Exemplo:

```
int numCols = rs.getMetaData().getColumnCount();
while (rs.next())
{ for (int col=1; col<=numCols; col++)
  { String st = rs.getString(col);
    System.out.print(st + "|");
  }
}
```


EXEMPLO - CÓDIGO SQL

```
ResultSet r = s.executeQuery(  
    "SELECT FIRST, LAST, EMAIL FROM people " +  
    "WHERE "(LAST="" + args[0] + "") " +  
    "AND (EMAIL Is Not Null) " +  
    "ORDER BY FIRST");  
while(r.next()) {  
    System.out.println( r.getString("Last") + ", "  
        + r.getString("fIRST")  
        + ": " + r.getString("EMAIL") );  
}
```

JDBC - GESTÃO

- ❑ Escrita na base de dados (insert, update, delete)

- ❑ executeUpdate

- ❑ Inserir um registo

```
String sql = "INSERT INTO Alunos VALUES
```

```
        ('12345','João Beirão', 'ET');
```

```
int count = stmt.executeUpdate(sql);
```

JDBC - EXEMPLO

- ❑ Criação de uma base de dados e ligação a um driver ODBC.
 - ❑ Criar uma base de dados, com o nome “**alunos**”, utilizando o **MS-Access**;
 - ❑ No Control Panel/ODBC/System DSN. Fazer:
 - 1) Faça Add, Microsoft Access Driver (*.mdb).
 - 2) Em Data Source Name escreva “alunos”
 - 3) Em Select seleccione o ficheiro criado em 1

JDBC - EXEMPLO

- ❑ A utilização de JDBC para criar tabelas, inserir, alterar e ler informação da base de dados, está exemplificada nos seguintes programas:
 - ❑ AlunosCreate.java,
 - ❑ AlunosInsert.java e
 - ❑ AlunosRetrieve.java
- ❑ Analisar e executar